Platform — Spark, Bud, and UNIBUD

One combined reference document containing all three layers, in dependency order: Spark (frozen intelligence engine) → Bud (frozen companion layer) → UNIBUD (the product being actively built on top of them).

This file is for reading/reference. It is not directly compilable — each `##` heading below gives the real file path; recreate that folder structure to actually build the project (or use the publish-ready package instead, which already has that structure).

Contents

Spark — Universal Intelligence System

README.md

````
# Spark — Universal Intelligence System

Spark is a standalone, product-agnostic intelligence platform. It is **not**
a chatbot, not a UI, and not tied to any single product. Products (Bud, a
future Realm Assistant, a future Orbit Assistant, etc.) consume Spark's
intelligence through one stable SDK boundary while keeping their own
personality and purpose.

```ts
import { createSpark } from "@/lib/spark";

const spark = createSpark();
await spark.initialize();

spark.identity.createContext({ userId: "u1", sessionId: "s1", product: "bud" });
const results = await spark.search.search("refund policy");
````
````

```
const plan = spark.planning.createPlan("Answer the user's question", ["Search kno
```

Consumers should only ever `import { ... } from "@/lib/spark"`. Nothing under `spark/core/*`, `spark/memory/*`, `spark/intelligence/*`, etc. is a supported import path — those are internal and may change.

Status

This is Phase 1: architecture, interfaces, and local (offline, no API key, no network call) implementations for every module. Real hosted model providers (OpenAI, Anthropic, Gemini) are wired as placeholder adapters behind the `AIProvider` interface but are not yet implemented — Spark runs entirely on the built-in `MockProvider` until you register a real one.

Verified in this repo:

- `npm run typecheck` **— passes with strict TypeScript**
- `npm run smoke-test` **— exercises all 18 services end to end**

Architecture

```
spark/
  container.ts        DI container (per-instance, lazy singletons)
  events.ts           Internal event bus (memory.updated, search.completed, ...)
  middleware.ts       Middleware pipeline (logging, auth, cache, rate-limit)
  plugins.ts          Plugin registration (extend Spark without modifying core)
  manifest.ts         Static metadata (version, capabilities)
  types.ts            Health/metrics types
  index.ts            Public SDK — the ONLY supported import path

  core/
    identity/         User, session, product, locale, timezone, device context
    reasoning/        Analysis and decisions, with explainable steps
    planning/         Goals, task sequencing, dependency-aware plans

  memory/             Short-term, long-term, semantic, episodic, workspace
  context/            Session/environment awareness (separate from identity)
  knowledge/          Document store + naive relevance query (swap for a
                      real knowledge graph / embeddings store later)

  intelligence/
    search/           Depends only on Knowledge (Search -> Knowledge)
    recommendations/  Tag-overlap scoring
```

```
  organization/            Grouping / sorting
  personalization/          Per-user preference store
  writing/                   Delegates to AIProvider
  translation/                 Delegates to AIProvider
  summaries/                    Deterministic extractive summarizer

trust/
  privacy/                PII detection + redaction (regex-based)
  security/                 Heuristic injection/scam-pattern scanner

automation/               Named task registration + execution + history
learning/                  Feedback capture + aggregate scoring

providers/
  types.ts                  AIProvider interface — the ONLY thing services call
  registry.ts                 Registration, default selection, availability fal
  mock.ts                       Default provider — deterministic, offline, alwa
  openai.ts / anthropic.ts /   Placeholder adapters. No SDK dependency, no netw
  gemini.ts / local.ts           calls. Implement complete() when ready to go l
```

Dependency direction

Services depend downward and never sideways in a way that would create cycles:

```
Search  ->  Knowledge
Writing / Translation / Reasoning  ->  ProviderRegistry (AIProvider interface)
```

Spark never imports any product (Bud, UNIBUD, My Realm, Orbit). Products only ever import `@/lib/spark`.

Lifecycle

```
await spark.initialize(); // resolves every registered service once, eagerly
await spark.reset();      // clears the container and re-registers core services
await spark.shutdown();   // clears event listeners and middleware
```

Introspection

```
spark.version();       // "0.1.0"
spark.modules();       // registered service tokens
spark.capabilities();  // ["identity", "reasoning", ..., "learning"]
spark.manifest();      // name, version, build, capabilities, modules, providers
```

```
spark.health();        // status, loaded modules, provider availability, diagnost
spark.metrics();       // memory size, events emitted (extend as real counters ar
```

Adding a real provider

```
import { createSpark, OpenAIProvider } from "@/lib/spark";

const spark = createSpark();
spark.registerProvider(new OpenAIProvider(process.env.OPENAI_API_KEY), /* makeDef
```

Until `OpenAIProvider.complete()` is implemented, `ProviderRegistry.resolve()` will keep silently falling back to `MockProvider` when the registered provider reports `isAvailable() === false` — Spark never throws just because a real provider isn't configured yet.

Extending Spark

- **Middleware —** `spark.use(loggingMiddleware)` or write your own `(ctx, next) => Promise<T>` function for auth, caching, rate limiting.
- **Plugins —** `spark.registerPlugin({ name, install(ctx) { ... } })` to add services, providers, or middleware without touching Spark's core.
- **Events —** `spark.events.on("search.completed", handler)`. Note: no service currently emits domain events yet; wire `events.emit(...)` calls into individual services as needed.

Explicitly out of scope for this phase

- No hosted model calls (all providers besides Mock throw until implemented)
- No persistence (everything is in-memory and resets on process restart)
- No product integration (Bud/UNIBUD/My Realm/Orbit are not referenced anywhere)
- No UI of any kind

````
## `package.json`

```json
{
````
````

```
  "name": "@ecosystem/spark",
  "version": "0.1.0",
  "description": "Spark — the Universal Intelligence System. Product-agnostic, pr
  "main": "src/lib/spark/index.ts",
  "types": "src/lib/spark/index.ts",
  "scripts": {
    "typecheck": "tsc --noEmit",
    "build": "tsc",
    "smoke-test": "ts-node scripts/smoke-test.ts"
  },
  "devDependencies": {
    "typescript": "^5.5.4",
    "ts-node": "^10.9.2",
    "@types/node": "^20.14.0"
  }
}
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "moduleResolution": "node",
    "ignoreDeprecations": "6.0",
    "lib": ["ES2020", "DOM"],
    "declaration": true,
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "baseUrl": ".",
    "paths": {
      "@/lib/spark": ["src/lib/spark/index.ts"],
      "@/lib/spark/*": ["src/lib/spark/*"]
    }
  },
  "include": ["src/**/*.ts", "scripts/**/*.ts"]
}
```

scripts/smoke-test.ts

```
import { createSpark, loggingMiddleware } from "../src/lib/spark";

async function main() {
  const spark = createSpark();
  spark.use(loggingMiddleware);
  await spark.initialize();

  console.log("=== manifest ===");
  console.log(JSON.stringify(spark.manifest(), null, 2));

  console.log("\n=== capabilities ===");
  console.log(spark.capabilities());

  // Identity
  const identity = spark.identity.createContext({
    userId: "user_1",
    sessionId: "session_1",
    product: "smoke-test",
  });
  console.log("\n=== identity ===", identity);

  // Memory
  spark.memory.remember({
    kind: "short_term",
    content: "The user asked about Spark's architecture.",
    sessionId: "session_1",
    userId: "user_1",
  });
  console.log("\n=== memory recall ===", spark.memory.recall({ sessionId: "sessio

  // Context
  spark.context.setAttribute("session_1", "currentScreen", "dashboard");
  console.log("\n=== context snapshot ===", spark.context.getSnapshot("session_1"

  // Knowledge + Search
  spark.knowledge.addDocument({
    title: "Spark Overview",
    content: "Spark is the Universal Intelligence System that powers products acr
    tags: ["spark", "architecture"],
  });
  const searchResults = await spark.search.search("universal intelligence");
  console.log("\n=== search ===", searchResults);

  // Reasoning
```

```
const reasoning = await spark.reasoning.analyze({
  question: "Should Bud call Spark or an AI provider directly?",
  facts: ["Bud must not contain intelligence or business logic."],
});
console.log("\n=== reasoning ===", reasoning);

// Planning
const plan = spark.planning.createPlan("Ship Spark v1", [
  "Define public API",
  "Implement local services",
  "Run production review",
]);
console.log("\n=== planning ===", plan, spark.planning.nextTask(plan.id));

// Recommendations
console.log(
  "\n=== recommendations ===",
  spark.recommendations.recommend({
    candidateItems: [
      { id: "a", tags: ["spark"] },
      { id: "b", tags: ["unrelated"] },
    ],
    basedOnTags: ["spark"],
  })
);

// Organization
console.log(
  "\n=== organization ===",
  spark.organization.groupByTag([
    { id: "1", label: "doc a", tags: ["spark"] },
    { id: "2", label: "doc b", tags: ["bud"] },
  ])
);

// Personalization
spark.personalization.setPreference("user_1", "theme", "dark");
console.log("\n=== personalization ===", spark.personalization.getAllPreference

// Writing / Translation / Summaries (mock provider)
console.log("\n=== writing ===", await spark.writing.draft({ prompt: "Say hello
console.log(
  "\n=== translation ===",
  await spark.translation.translate({ text: "Hello", targetLocale: "es-ES" })
);
console.log(
  "\n=== summaries ===",
```

```
    spark.summaries.summarize({
      text: "Spark is the Universal Intelligence System. It powers Bud, Realm Ass
      maxSentences: 1,
    })
  );

  // Privacy / Security
  console.log(
    "\n=== privacy scan ===",
    spark.privacy.scanForPii("Contact me at test@example.com")
  );
  console.log(
    "\n=== security check ===",
    spark.security.checkInput("Please verify your password immediately")
  );

  // Automation
  spark.automation.registerTask("ping", async () => "pong");
  console.log("\n=== automation ===", await spark.automation.run("ping"));

  // Learning
  spark.learning.recordFeedback({ subject: "writing", rating: "positive" });
  console.log("\n=== learning score ===", spark.learning.score("writing"));

  // Health
  console.log("\n=== health ===", spark.health());

  await spark.shutdown();
  console.log("\nSmoke test completed successfully.");
}

main().catch((err) => {
  console.error("Smoke test failed:", err);
  throw err;
});
```

scripts/container-verify.ts

```
import { createSpark } from "../src/lib/spark";
import { Container } from "../src/lib/spark/container";
import { ServiceNotRegisteredError } from "../src/lib/spark/errors";

async function main() {
  const spark = createSpark();
```

```
  console.log("modules():", spark.modules());
  console.log("manifest().registeredModules:", spark.manifest().registeredModules

  // registerValue + missing-token error behavior, isolated from Spark's own cont
  const c = new Container();
  const sentinel = { ok: true };
  const token = Symbol("test.value");
  c.registerValue(token, sentinel);
  const resolved = c.resolve<typeof sentinel>(token);
  if (resolved !== sentinel) throw new Error("registerValue did not preserve iden
  console.log("registerValue OK, resolved instance === original:", resolved === s

  try {
    c.resolve(Symbol("never.registered"));
    throw new Error("expected ServiceNotRegisteredError to be thrown");
  } catch (err) {
    if (!(err instanceof ServiceNotRegisteredError)) {
      throw new Error(`expected ServiceNotRegisteredError, got ${err}`);
    }
    console.log("ServiceNotRegisteredError thrown correctly:", err.message);
  }

  // Per-instance isolation: two Spark instances never share a container
  const sparkA = createSpark();
  const sparkB = createSpark();
  sparkA.memory.remember({ kind: "short_term", content: "only in A" });
  const crossLeak = sparkB.memory.recall({ text: "only in A" });
  if (crossLeak.length !== 0) throw new Error("Spark instances are leaking state!
  console.log("Per-instance isolation OK: sparkB sees", crossLeak.length, "matchi

  console.log("\nContainer verification completed successfully.");
}

main().catch((err) => {
  console.error("Container verification failed:", err);
  throw err;
});
```

src/lib/spark/index.ts

```
/**
 * Public Spark SDK.
 *
 * Consumers should only ever import from here:
```

```
 *
 *   import { createSpark } from "@/lib/spark";
 *
 * Never import from spark/core/*, spark/memory/*, etc. directly —
 * those paths are internal implementation detail and may change.
 */

import { Container } from "./container";
import { TOKENS, tokenLabel } from "./tokens";
import { EventBus } from "./events";
import { MiddlewarePipeline, type Middleware } from "./middleware";
import { PluginManager, type SparkPlugin } from "./plugins";
import { ProviderRegistry } from "./providers/registry";
import type { AIProvider } from "./providers/types";
import {
  SPARK_CAPABILITIES,
  SPARK_BUILD,
  SPARK_VERSION,
  type SparkManifest,
} from "./manifest";
import type { SparkHealthReport, SparkMetrics } from "./types";

import { LocalIdentityService } from "./core/identity/local";
import { LocalReasoningService } from "./core/reasoning/local";
import { LocalPlanningService } from "./core/planning/local";
import { InMemoryMemoryService } from "./memory/local";
import { LocalContextService } from "./context/local";
import { LocalKnowledgeService } from "./knowledge/local";
import { LocalSearchService } from "./intelligence/search/local";
import { LocalRecommendationsService } from "./intelligence/recommendations/local
import { LocalOrganizationService } from "./intelligence/organization/local";
import { LocalPersonalizationService } from "./intelligence/personalization/local
import { LocalWritingService } from "./intelligence/writing/local";
import { LocalTranslationService } from "./intelligence/translation/local";
import { LocalSummariesService } from "./intelligence/summaries/local";
import { LocalPrivacyService } from "./trust/privacy/local";
import { LocalSecurityService } from "./trust/security/local";
import { LocalAutomationService } from "./automation/local";
import { LocalLearningService } from "./learning/local";

import type { IdentityService } from "./core/identity/interface";
import type { ReasoningService } from "./core/reasoning/interface";
import type { PlanningService } from "./core/planning/interface";
import type { MemoryService } from "./memory/interface";
import type { ContextService } from "./context/interface";
import type { KnowledgeService } from "./knowledge/interface";
import type { SearchService } from "./intelligence/search/interface";
```

```
import type { RecommendationsService } from "./intelligence/recommendations/inter
import type { OrganizationService } from "./intelligence/organization/interface";
import type { PersonalizationService } from "./intelligence/personalization/inter
import type { WritingService } from "./intelligence/writing/interface";
import type { TranslationService } from "./intelligence/translation/interface";
import type { SummariesService } from "./intelligence/summaries/interface";
import type { PrivacyService } from "./trust/privacy/interface";
import type { SecurityService } from "./trust/security/interface";
import type { AutomationService } from "./automation/interface";
import type { LearningService } from "./learning/interface";

export class Spark {
  readonly events = new EventBus();
  readonly middleware = new MiddlewarePipeline();

  private readonly container = new Container();
  private readonly providerRegistry = new ProviderRegistry();
  private readonly plugins = new PluginManager();
  private readonly startedAt = Date.now();
  private initialized = false;
  private readonly warnings: string[] = [];

  constructor() {
    this.registerCoreServices();
  }

  // ---------------------------------------------------------------------
  // Lifecycle
  // ---------------------------------------------------------------------

  async initialize(): Promise<void> {
    if (this.initialized) return;
    // Eagerly resolve every registered service once so configuration
    // errors surface immediately rather than on first use.
    for (const token of this.container.registeredTokens()) {
      this.container.resolve(token);
    }
    this.initialized = true;
    this.events.emit("spark.initialized", { at: new Date().toISOString() });
  }

  async shutdown(): Promise<void> {
    this.events.emit("spark.shutdown", { at: new Date().toISOString() });
    this.events.clear();
    this.middleware.clear();
    this.initialized = false;
  }
```

```
async reset(): Promise<void> {
  this.container.reset();
  this.registerCoreServices();
  this.initialized = false;
}

// ------------------------------------------------------------------------
// Providers & plugins
// ------------------------------------------------------------------------

registerProvider(provider: AIProvider, makeDefault = false): void {
  this.providerRegistry.register(provider, makeDefault);
}

registerPlugin(plugin: SparkPlugin): void {
  this.plugins.register(plugin, {
    container: this.container,
    providers: this.providerRegistry,
    middleware: this.middleware,
    events: this.events,
  });
}

use(middleware: Middleware): void {
  this.middleware.use(middleware);
}

// ------------------------------------------------------------------------
// Introspection
// ------------------------------------------------------------------------

version(): string {
  return SPARK_VERSION;
}

modules(): string[] {
  return this.container.registeredTokens().map(tokenLabel);
}

capabilities(): readonly string[] {
  return SPARK_CAPABILITIES;
}

manifest(): SparkManifest {
  return {
    name: "Spark",
```

```
    version: SPARK_VERSION,
    build: SPARK_BUILD,
    capabilities: SPARK_CAPABILITIES,
    registeredModules: this.container.registeredTokens().map(tokenLabel),
    providers: this.providerRegistry.list(),
  };
}

health(): SparkHealthReport {
  const diagnostics: string[] = [];
  const providers = this.providerRegistry.list();

  if (!providers.some((p) => p.available)) {
    this.warnings.push(
      "No AI provider is currently available besides the mock provider."
    );
  }

  diagnostics.push(
    `${this.container.resolvedTokens().length}/${
      this.container.registeredTokens().length
    } services resolved.`
  );

  const status: SparkHealthReport["status"] = this.initialized
    ? "healthy"
    : "degraded";

  return {
    status,
    initialized: this.initialized,
    loadedModules: this.container.resolvedTokens().map(tokenLabel),
    registeredProviders: providers,
    diagnostics,
    warnings: [...new Set(this.warnings)],
    uptimeMs: Date.now() - this.startedAt,
    checkedAt: new Date().toISOString(),
  };
}

metrics(): SparkMetrics {
  const events = this.events.recentEvents(500);
  return {
    requestCounts: {},
    executionTimesMs: {},
    cacheHits: 0,
    cacheMisses: 0,
```

```
    memorySize: this.memory.size(),
    eventsEmitted: events.length,
  };
}

// ---------------------------------------------------------------------
// Service accessors — this is the actual public surface products use
// ---------------------------------------------------------------------

get identity(): IdentityService {
  return this.container.resolve(TOKENS.Identity);
}

get reasoning(): ReasoningService {
  return this.container.resolve(TOKENS.Reasoning);
}

get planning(): PlanningService {
  return this.container.resolve(TOKENS.Planning);
}

get memory(): MemoryService {
  return this.container.resolve(TOKENS.Memory);
}

get context(): ContextService {
  return this.container.resolve(TOKENS.Context);
}

get knowledge(): KnowledgeService {
  return this.container.resolve(TOKENS.Knowledge);
}

get search(): SearchService {
  return this.container.resolve(TOKENS.Search);
}

get recommendations(): RecommendationsService {
  return this.container.resolve(TOKENS.Recommendations);
}

get organization(): OrganizationService {
  return this.container.resolve(TOKENS.Organization);
}

get personalization(): PersonalizationService {
  return this.container.resolve(TOKENS.Personalization);
```

```
}

get writing(): WritingService {
  return this.container.resolve(TOKENS.Writing);
}

get translation(): TranslationService {
  return this.container.resolve(TOKENS.Translation);
}

get summaries(): SummariesService {
  return this.container.resolve(TOKENS.Summaries);
}

get privacy(): PrivacyService {
  return this.container.resolve(TOKENS.Privacy);
}

get security(): SecurityService {
  return this.container.resolve(TOKENS.Security);
}

get automation(): AutomationService {
  return this.container.resolve(TOKENS.Automation);
}

get learning(): LearningService {
  return this.container.resolve(TOKENS.Learning);
}

// ---------------------------------------------------------------------
// Internal wiring
// ---------------------------------------------------------------------

private registerCoreServices(): void {
  this.container.registerValue(TOKENS.ProviderRegistry, this.providerRegistry);

  this.container.register(TOKENS.Identity, () => new LocalIdentityService());
  this.container.register(
    TOKENS.Reasoning,
    () => new LocalReasoningService(this.providerRegistry)
  );
  this.container.register(TOKENS.Planning, () => new LocalPlanningService());
  this.container.register(TOKENS.Memory, () => new InMemoryMemoryService());
  this.container.register(TOKENS.Context, () => new LocalContextService());
  this.container.register(TOKENS.Knowledge, () => new LocalKnowledgeService());
```

```
    // Search depends on Knowledge only — Search -> Knowledge, not Memory,
    // to keep the dependency graph shallow as Spark grows.
    this.container.register(
      TOKENS.Search,
      (c) => new LocalSearchService(c.resolve(TOKENS.Knowledge))
    );

    this.container.register(
      TOKENS.Recommendations,
      () => new LocalRecommendationsService()
    );
    this.container.register(
      TOKENS.Organization,
      () => new LocalOrganizationService()
    );
    this.container.register(
      TOKENS.Personalization,
      () => new LocalPersonalizationService()
    );
    this.container.register(
      TOKENS.Writing,
      () => new LocalWritingService(this.providerRegistry)
    );
    this.container.register(
      TOKENS.Translation,
      () => new LocalTranslationService(this.providerRegistry)
    );
    this.container.register(TOKENS.Summaries, () => new LocalSummariesService());
    this.container.register(TOKENS.Privacy, () => new LocalPrivacyService());
    this.container.register(TOKENS.Security, () => new LocalSecurityService());
    this.container.register(TOKENS.Automation, () => new LocalAutomationService()
    this.container.register(TOKENS.Learning, () => new LocalLearningService());
  }
}

/**
 * Factory for a new, fully independent Spark instance. Each call
 * produces its own container/providers/events/middleware — instances
 * never share state.
 */
export function createSpark(): Spark {
  return new Spark();
}

// Re-export types consumers are expected to use.
export type { AIProvider } from "./providers/types";
export type { SparkPlugin, SparkPluginContext } from "./plugins";
```

```
export type { Middleware } from "./middleware";
export type { SparkManifest } from "./manifest";
export type { SparkHealthReport, SparkMetrics } from "./types";
export { loggingMiddleware } from "./middleware";
export { MockProvider } from "./providers/mock";
export { OpenAIProvider } from "./providers/openai";
export { AnthropicProvider } from "./providers/anthropic";
export { GeminiProvider } from "./providers/gemini";
export { LocalModelProvider } from "./providers/local";
```

src/lib/spark/container.ts

```
/**
 * Minimal dependency-injection container.
 *
 * Services are registered as factories and resolved lazily on first
 * use, then cached as singletons for the lifetime of the Spark instance
 * that owns this container. Each `createSpark()` call constructs its
 * own Container, so multiple Spark instances never share state — there
 * is no module-level/global registry anywhere in this file.
 *
 * Canonical tokens live in ./tokens.ts — this file only consumes them.
 */

import type { Token } from "./tokens";
import { tokenLabel } from "./tokens";
import { ServiceNotRegisteredError } from "./errors";

type Factory<T> = (container: Container) => T;

export class Container {
  private factories = new Map<Token, Factory<unknown>>();
  private instances = new Map<Token, unknown>();

  /** Register a lazy factory. Only invoked on first resolve(). */
  register<T>(token: Token, factory: Factory<T>): void {
    this.factories.set(token, factory as Factory<unknown>);
    // Invalidate any cached instance so re-registration is respected.
    this.instances.delete(token);
  }

  /**
   * Register an already-constructed value directly (e.g. a shared
   * Logger or ProviderRegistry instance) without wrapping it in a
```

```
   * factory function. Still resolved lazily/cached like any other
   * token — resolve() doesn't distinguish how a token was registered.
   */
  registerValue<T>(token: Token, value: T): void {
    this.register(token, () => value);
  }

  has(token: Token): boolean {
    return this.factories.has(token);
  }

  resolve<T>(token: Token): T {
    if (this.instances.has(token)) {
      return this.instances.get(token) as T;
    }
    const factory = this.factories.get(token);
    if (!factory) {
      throw new ServiceNotRegisteredError(tokenLabel(token));
    }
    const instance = factory(this);
    this.instances.set(token, instance);
    return instance as T;
  }

  /** Tokens that currently have a registered factory. */
  registeredTokens(): Token[] {
    return Array.from(this.factories.keys());
  }

  /** Tokens that have actually been instantiated (lazy resolution check). */
  resolvedTokens(): Token[] {
    return Array.from(this.instances.keys());
  }

  reset(): void {
    this.instances.clear();
  }
}
```

src/lib/spark/tokens.ts

```
/**
 * The single canonical set of DI tokens used throughout Spark.
 *
```

```
 * Tokens are Symbols, not strings, so two independently-authored
 * modules can never collide on a token by accident even if they
 * happen to choose the same label. Every Symbol still carries its
 * original label as `description`, so diagnostics and logs stay
 * human-readable via `tokenLabel()` below.
 */

export const TOKENS = {
  Logger: Symbol("kernel.logger"),
  ProviderRegistry: Symbol("providers.registry"),

  Identity: Symbol("core.identity"),
  Memory: Symbol("core.memory"),
  Context: Symbol("core.context"),
  Knowledge: Symbol("core.knowledge"),
  Reasoning: Symbol("core.reasoning"),
  Planning: Symbol("core.planning"),

  Search: Symbol("intelligence.search"),
  Recommendations: Symbol("intelligence.recommendations"),
  Organization: Symbol("intelligence.organization"),
  Personalization: Symbol("intelligence.personalization"),
  Writing: Symbol("intelligence.writing"),
  Translation: Symbol("intelligence.translation"),
  Summaries: Symbol("intelligence.summaries"),

  Privacy: Symbol("trust.privacy"),
  Security: Symbol("trust.security"),

  Automation: Symbol("automation"),
  Learning: Symbol("learning"),
} as const;

export type Token = symbol;

/** Human-readable label for a token, for logs/diagnostics/health output. */
export function tokenLabel(token: Token): string {
  return token.description ?? token.toString();
}
```

src/lib/spark/errors.ts

```
/**
 * Spark's error hierarchy. Internal code should throw these instead of
```

```
 * generic `Error` so callers (and Kernel diagnostics) can distinguish
 * failure classes reliably.
 */

export class SparkError extends Error {
  constructor(message: string) {
    super(message);
    this.name = new.target.name;
    Object.setPrototypeOf(this, new.target.prototype);
  }
}

/** Thrown when Container.resolve() is called for an unregistered token. */
export class ServiceNotRegisteredError extends SparkError {
  constructor(public readonly tokenLabel: string) {
    super(`No service registered for token "${tokenLabel}".`);
  }
}

/** Thrown when a provider is looked up by name but was never registered. */
export class ProviderNotRegisteredError extends SparkError {
  constructor(public readonly providerName: string) {
    super(`Provider "${providerName}" is not registered.`);
  }
}

/** Thrown when a provider is resolved but cannot currently be used (e.g. no cred
export class ProviderUnavailableError extends SparkError {
  constructor(public readonly providerName: string, reason?: string) {
    super(
      `Provider "${providerName}" is not available${reason ? `: ${reason}` : "."}
    );
  }
}

/** Thrown by services when given malformed input. */
export class ValidationError extends SparkError {
  constructor(message: string) {
    super(message);
  }
}

/** Thrown when an operation requires Spark to be initialized first. */
export class NotInitializedError extends SparkError {
  constructor() {
    super("Spark has not been initialized. Call initialize() first.");
  }
```

```
}

/** Thrown when a plugin name is registered more than once. */
export class PluginAlreadyRegisteredError extends SparkError {
  constructor(public readonly pluginName: string) {
    super(`Plugin "${pluginName}" is already registered.`);
  }
}
```

src/lib/spark/events.ts

```
/**
 * Lightweight internal event bus.
 *
 * Modules publish events instead of calling each other directly, which
 * keeps them decoupled. External consumers (e.g. Oracle) can subscribe
 * without Spark needing to know they exist.
 *
 * Example event names: "memory.updated", "search.completed",
 * "recommendation.generated", "translation.finished".
 */

export type SparkEventName = string;

export type SparkEventHandler<TPayload = unknown> = (
  payload: TPayload
) => void;

export interface SparkEventLogEntry {
  name: SparkEventName;
  payload: unknown;
  timestamp: string;
}

export class EventBus {
  private handlers = new Map<SparkEventName, Set<SparkEventHandler>>();
  private log: SparkEventLogEntry[] = [];
  private readonly maxLog = 200;

  on<TPayload = unknown>(
    name: SparkEventName,
    handler: SparkEventHandler<TPayload>
  ): () => void {
    if (!this.handlers.has(name)) {
```

```
      this.handlers.set(name, new Set());
    }
    this.handlers.get(name)!.add(handler as SparkEventHandler);
    return () => this.off(name, handler);
  }

  off<TPayload = unknown>(
    name: SparkEventName,
    handler: SparkEventHandler<TPayload>
  ): void {
    this.handlers.get(name)?.delete(handler as SparkEventHandler);
  }

  emit<TPayload = unknown>(name: SparkEventName, payload?: TPayload): void {
    this.log.push({
      name,
      payload,
      timestamp: new Date().toISOString(),
    });
    if (this.log.length > this.maxLog) {
      this.log.shift();
    }
    this.handlers.get(name)?.forEach((handler) => {
      try {
        handler(payload);
      } catch (err) {
        // Event handlers must never break the emitting module.
        // eslint-disable-next-line no-console
        console.error(`[spark:events] handler for "${name}" threw`, err);
      }
    });
  }

  recentEvents(limit = 50): SparkEventLogEntry[] {
    return this.log.slice(-limit);
  }

  clear(): void {
    this.handlers.clear();
    this.log = [];
  }
}
```

src/lib/spark/middleware.ts

```
/**
 * Middleware pipeline.
 *
 * Middleware wraps every call made through Spark.call(), so cross-cutting
 * concerns (logging, auth, caching, rate limiting) can be added without
 * touching individual service implementations.
 */

export interface SparkCallContext {
  service: string;
  method: string;
  args: unknown[];
  startedAt: number;
  meta: Record<string, unknown>;
}

export type NextFn<TResult> = () => Promise<TResult>;

export type Middleware = <TResult>(
  ctx: SparkCallContext,
  next: NextFn<TResult>
) => Promise<TResult>;

export class MiddlewarePipeline {
  private middlewares: Middleware[] = [];

  use(middleware: Middleware): void {
    this.middlewares.push(middleware);
  }

  clear(): void {
    this.middlewares = [];
  }

  list(): number {
    return this.middlewares.length;
  }

  async run<TResult>(
    ctx: SparkCallContext,
    handler: () => Promise<TResult>
  ): Promise<TResult> {
    const chain = this.middlewares.reduceRight<NextFn<TResult>>(
      (next, mw) => () => mw(ctx, next),
      handler
    );
```

```
    return chain();
  }
}

/** Simple built-in logging middleware, opt-in via Spark.use(loggingMiddleware).
export const loggingMiddleware: Middleware = async (ctx, next) => {
  const result = await next();
  const durationMs = Date.now() - ctx.startedAt;
  // eslint-disable-next-line no-console
  console.log(
    `[spark] ${ctx.service}.${ctx.method} (${durationMs}ms)`
  );
  return result;
};
```

src/lib/spark/plugins.ts

```
import type { Container } from "./container";
import type { Token } from "./tokens";
import type { ProviderRegistry } from "./providers/registry";
import type { MiddlewarePipeline } from "./middleware";
import type { EventBus } from "./events";

export interface SparkPluginContext {
  container: Container;
  providers: ProviderRegistry;
  middleware: MiddlewarePipeline;
  events: EventBus;
}

export interface SparkPlugin {
  name: string;
  version?: string;
  /** Called once at registration time. Register services, providers, etc. here.
  install(ctx: SparkPluginContext): void;
}

export class PluginManager {
  private installed = new Map<string, SparkPlugin>();

  register(plugin: SparkPlugin, ctx: SparkPluginContext): void {
    if (this.installed.has(plugin.name)) {
      throw new Error(`Plugin "${plugin.name}" is already registered.`);
    }
```

```
    plugin.install(ctx);
    this.installed.set(plugin.name, plugin);
  }

  list(): Array<{ name: string; version?: string }> {
    return Array.from(this.installed.values()).map((p) => ({
      name: p.name,
      version: p.version,
    }));
  }
}

// Re-export for convenience so plugin authors have one import.
export type { Token };
```

src/lib/spark/manifest.ts

```
export const SPARK_VERSION = "0.1.0";
export const SPARK_BUILD = "dev";

export const SPARK_CAPABILITIES = [
  "identity",
  "reasoning",
  "planning",
  "memory",
  "context",
  "knowledge",
  "search",
  "recommendations",
  "organization",
  "personalization",
  "writing",
  "translation",
  "summaries",
  "privacy",
  "security",
  "automation",
  "learning",
] as const;

export type SparkCapability = (typeof SPARK_CAPABILITIES)[number];

export interface SparkManifest {
  name: string;
```

```
  version: string;
  build: string;
  capabilities: readonly SparkCapability[];
  registeredModules: string[];
  providers: Array<{ name: string; available: boolean; isDefault: boolean }>;
}
```

src/lib/spark/types.ts

```
export type HealthStatus = "healthy" | "degraded" | "unhealthy";

export interface SparkHealthReport {
  status: HealthStatus;
  initialized: boolean;
  loadedModules: string[];
  registeredProviders: Array<{
    name: string;
    available: boolean;
    isDefault: boolean;
  }>;
  diagnostics: string[];
  warnings: string[];
  uptimeMs: number;
  checkedAt: string;
}

export interface SparkMetrics {
  requestCounts: Record<string, number>;
  executionTimesMs: Record<string, number[]>;
  cacheHits: number;
  cacheMisses: number;
  memorySize: number;
  eventsEmitted: number;
}
```

src/lib/spark/core/identity/interface.ts

```
export interface DeviceContext {
  type?: "desktop" | "mobile" | "tablet" | "server" | "unknown";
  os?: string;
}
```

```
export interface IdentityContext {
  userId: string;
  sessionId: string;
  product: string;
  locale: string;
  timezone: string;
  device?: DeviceContext;
  createdAt: string;
}

export interface IdentityService {
  /** Create (or overwrite) the identity context for a session. */
  createContext(input: {
    userId: string;
    sessionId: string;
    product: string;
    locale?: string;
    timezone?: string;
    device?: DeviceContext;
  }): IdentityContext;

  getContext(sessionId: string): IdentityContext | undefined;

  updateContext(
    sessionId: string,
    patch: Partial<Omit<IdentityContext, "sessionId" | "createdAt">>
  ): IdentityContext | undefined;

  clearContext(sessionId: string): boolean;

  /** Number of active identity contexts currently tracked. */
  size(): number;
}
```

src/lib/spark/core/identity/local.ts

```
import type { IdentityContext, IdentityService, DeviceContext } from "./interface

/**
 * In-memory identity service. Provides user/session/product/locale/
 * timezone/device context for all other Spark services. No persistence
 * or network calls — state lives for the lifetime of the Spark instance.
 */
```

```
export class LocalIdentityService implements IdentityService {
  private contexts = new Map<string, IdentityContext>();

  createContext(input: {
    userId: string;
    sessionId: string;
    product: string;
    locale?: string;
    timezone?: string;
    device?: DeviceContext;
  }): IdentityContext {
    const context: IdentityContext = {
      userId: input.userId,
      sessionId: input.sessionId,
      product: input.product,
      locale: input.locale ?? "en-US",
      timezone: input.timezone ?? "UTC",
      device: input.device,
      createdAt: new Date().toISOString(),
    };
    this.contexts.set(input.sessionId, context);
    return context;
  }

  getContext(sessionId: string): IdentityContext | undefined {
    return this.contexts.get(sessionId);
  }

  updateContext(
    sessionId: string,
    patch: Partial<Omit<IdentityContext, "sessionId" | "createdAt">>
  ): IdentityContext | undefined {
    const existing = this.contexts.get(sessionId);
    if (!existing) return undefined;
    const updated = { ...existing, ...patch };
    this.contexts.set(sessionId, updated);
    return updated;
  }

  clearContext(sessionId: string): boolean {
    return this.contexts.delete(sessionId);
  }

  size(): number {
    return this.contexts.size;
  }
}
```

src/lib/spark/core/reasoning/interface.ts

```
export interface ReasoningInput {
  question: string;
  facts?: string[];
  context?: Record<string, unknown>;
}

export interface ReasoningStep {
  step: number;
  description: string;
}

export interface ReasoningResult {
  answer: string;
  confidence: number; // 0..1
  steps: ReasoningStep[];
  usedProvider: string;
}

export interface ReasoningService {
  /** Analyze input and produce a decision/answer with explainable steps. */
  analyze(input: ReasoningInput): Promise<ReasoningResult>;
}
```

src/lib/spark/core/reasoning/local.ts

```
import type {
  ReasoningInput,
  ReasoningResult,
  ReasoningService,
} from "./interface";
import type { ProviderRegistry } from "../../providers/registry";

/**
 * Deterministic local reasoning implementation. Produces a transparent,
 * step-by-step trace without requiring a hosted model. When a real
 * provider becomes available, this class can delegate to it through
 * ProviderRegistry without callers changing anything.
 */
```

```
export class LocalReasoningService implements ReasoningService {
  constructor(private readonly providers: ProviderRegistry) {}

  async analyze(input: ReasoningInput): Promise<ReasoningResult> {
    const provider = this.providers.resolve();
    const facts = input.facts ?? [];

    const steps = [
      { step: 1, description: `Parsed question: "${input.question}"` },
      {
        step: 2,
        description: facts.length
          ? `Considered ${facts.length} supplied fact(s).`
          : "No supporting facts were supplied.",
      },
      {
        step: 3,
        description: `Delegated synthesis to provider "${provider.name}".`,
      },
    ];

    const completion = await provider.complete({
      messages: [
        { role: "system", content: "You are Spark's reasoning module." },
        {
          role: "user",
          content: `Question: ${input.question}\nFacts: ${facts.join("; ")}`,
        },
      ],
    });

    return {
      answer: completion.text,
      confidence: facts.length > 0 ? 0.6 : 0.35,
      steps,
      usedProvider: provider.name,
    };
  }
}
```

src/lib/spark/core/planning/interface.ts

```
export interface PlanTask {
  id: string;
```

```
  title: string;
  done: boolean;
  order: number;
  dependsOn?: string[];
}

export interface Plan {
  id: string;
  goal: string;
  tasks: PlanTask[];
  createdAt: string;
}

export interface PlanningService {
  createPlan(goal: string, taskTitles: string[]): Plan;
  getPlan(planId: string): Plan | undefined;
  completeTask(planId: string, taskId: string): Plan | undefined;
  nextTask(planId: string): PlanTask | undefined;
}
```

src/lib/spark/core/planning/local.ts

```
import type { Plan, PlanTask, PlanningService } from "./interface";

/**
 * In-memory planning service. Handles goal decomposition into ordered,
 * dependency-aware tasks. No AI provider is required for the core
 * sequencing logic; reasoning-assisted decomposition can be layered on
 * top later via the ReasoningService.
 */
export class LocalPlanningService implements PlanningService {
  private plans = new Map<string, Plan>();
  private counter = 0;

  createPlan(goal: string, taskTitles: string[]): Plan {
    const id = `plan_${++this.counter}_${Date.now()}`;
    const tasks: PlanTask[] = taskTitles.map((title, index) => ({
      id: `${id}_task_${index}`,
      title,
      done: false,
      order: index,
      dependsOn: index > 0 ? [`${id}_task_${index - 1}`] : [],
    }));
    const plan: Plan = { id, goal, tasks, createdAt: new Date().toISOString() };
```

```
    this.plans.set(id, plan);
    return plan;
  }

  getPlan(planId: string): Plan | undefined {
    return this.plans.get(planId);
  }

  completeTask(planId: string, taskId: string): Plan | undefined {
    const plan = this.plans.get(planId);
    if (!plan) return undefined;
    const task = plan.tasks.find((t) => t.id === taskId);
    if (task) task.done = true;
    return plan;
  }

  nextTask(planId: string): PlanTask | undefined {
    const plan = this.plans.get(planId);
    if (!plan) return undefined;
    return plan.tasks
      .filter((t) => !t.done)
      .sort((a, b) => a.order - b.order)
      .find((t) =>
        (t.dependsOn ?? []).every(
          (depId) => plan.tasks.find((d) => d.id === depId)?.done
        )
      );
  }
}
```

src/lib/spark/memory/interface.ts

```
export type MemoryKind = "short_term" | "long_term" | "semantic" | "episodic" | "

export interface MemoryRecord {
  id: string;
  kind: MemoryKind;
  sessionId?: string;
  userId?: string;
  content: string;
  tags?: string[];
  createdAt: string;
  /**
   * Monotonically increasing insertion order. `createdAt` alone is not a
```

```
   * reliable sort key — multiple records can share the same millisecond
   * timestamp. `recall()` implementations must use `sequence` as the
   * tiebreaker so "newest first" holds even for same-millisecond writes.
   */
  sequence: number;
}

export interface MemoryQuery {
  kind?: MemoryKind;
  sessionId?: string;
  userId?: string;
  tag?: string;
  text?: string;
  limit?: number;
}

export interface MemoryService {
  remember(input: {
    kind: MemoryKind;
    content: string;
    sessionId?: string;
    userId?: string;
    tags?: string[];
  }): MemoryRecord;

  recall(query: MemoryQuery): MemoryRecord[];

  forget(id: string): boolean;

  clearSession(sessionId: string): number;

  size(): number;
}
```

src/lib/spark/memory/local.ts

```
import type { MemoryKind, MemoryQuery, MemoryRecord, MemoryService } from "./inte

/**
 * In-memory implementation covering short-term, long-term, semantic,
 * episodic and workspace memory kinds through a single flat store with
 * a `kind` discriminator. No persistence, no network calls.
 */
export class InMemoryMemoryService implements MemoryService {
```

```
private records = new Map<string, MemoryRecord>();
private counter = 0;

remember(input: {
  kind: MemoryKind;
  content: string;
  sessionId?: string;
  userId?: string;
  tags?: string[];
}): MemoryRecord {
  const sequence = ++this.counter;
  const record: MemoryRecord = {
    id: `mem_${sequence}_${Date.now()}`,
    kind: input.kind,
    sessionId: input.sessionId,
    userId: input.userId,
    content: input.content,
    tags: input.tags ?? [],
    createdAt: new Date().toISOString(),
    sequence,
  };
  this.records.set(record.id, record);
  return record;
}

recall(query: MemoryQuery): MemoryRecord[] {
  let results = Array.from(this.records.values());

  if (query.kind) results = results.filter((r) => r.kind === query.kind);
  if (query.sessionId)
    results = results.filter((r) => r.sessionId === query.sessionId);
  if (query.userId) results = results.filter((r) => r.userId === query.userId);
  if (query.tag) results = results.filter((r) => (r.tags ?? []).includes(query.
  if (query.text) {
    const needle = query.text.toLowerCase();
    results = results.filter((r) => r.content.toLowerCase().includes(needle));
  }

  results.sort((a, b) => {
    const timeDiff = new Date(b.createdAt).getTime() - new Date(a.createdAt).ge
    return timeDiff !== 0 ? timeDiff : b.sequence - a.sequence;
  });

  return query.limit ? results.slice(0, query.limit) : results;
}

forget(id: string): boolean {
```

```
    return this.records.delete(id);
  }

  clearSession(sessionId: string): number {
    let cleared = 0;
    for (const [id, record] of this.records) {
      if (record.sessionId === sessionId) {
        this.records.delete(id);
        cleared++;
      }
    }
    return cleared;
  }

  size(): number {
    return this.records.size;
  }
}
```

src/lib/spark/context/interface.ts

```
export interface EnvironmentContext {
  platform?: string;
  appVersion?: string;
  networkQuality?: "offline" | "poor" | "good" | "excellent";
}

export interface SessionContextSnapshot {
  sessionId: string;
  product: string;
  environment?: EnvironmentContext;
  attributes: Record<string, unknown>;
  updatedAt: string;
}

export interface ContextService {
  setEnvironment(sessionId: string, env: EnvironmentContext): void;
  setAttribute(sessionId: string, key: string, value: unknown): void;
  getSnapshot(sessionId: string, product: string): SessionContextSnapshot;
  clear(sessionId: string): boolean;
}
```

src/lib/spark/context/local.ts

```
import type {
  ContextService,
  EnvironmentContext,
  SessionContextSnapshot,
} from "./interface";

/**
 * Tracks per-session environmental/product awareness (in addition to
 * the user/session identity owned by the Identity service). Kept
 * separate from Identity because context changes far more often
 * (network quality, current screen, ephemeral attributes) than
 * identity does.
 */
export class LocalContextService implements ContextService {
  private environments = new Map<string, EnvironmentContext>();
  private attributes = new Map<string, Record<string, unknown>>();

  setEnvironment(sessionId: string, env: EnvironmentContext): void {
    this.environments.set(sessionId, env);
  }

  setAttribute(sessionId: string, key: string, value: unknown): void {
    const existing = this.attributes.get(sessionId) ?? {};
    existing[key] = value;
    this.attributes.set(sessionId, existing);
  }

  getSnapshot(sessionId: string, product: string): SessionContextSnapshot {
    return {
      sessionId,
      product,
      environment: this.environments.get(sessionId),
      attributes: this.attributes.get(sessionId) ?? {},
      updatedAt: new Date().toISOString(),
    };
  }

  clear(sessionId: string): boolean {
    const hadEnv = this.environments.delete(sessionId);
    const hadAttrs = this.attributes.delete(sessionId);
    return hadEnv || hadAttrs;
  }
}
```

src/lib/spark/knowledge/interface.ts

```
export interface KnowledgeDocument {
  id: string;
  title: string;
  content: string;
  tags?: string[];
  addedAt: string;
}

export interface KnowledgeQueryResult {
  document: KnowledgeDocument;
  score: number;
}

export interface KnowledgeService {
  addDocument(input: { title: string; content: string; tags?: string[] }): Knowle
  getDocument(id: string): KnowledgeDocument | undefined;
  removeDocument(id: string): boolean;
  /** Simple relevance query. Real embeddings can replace scoring later. */
  query(text: string, limit?: number): KnowledgeQueryResult[];
  size(): number;
}
```

src/lib/spark/knowledge/local.ts

```
import type {
  KnowledgeDocument,
  KnowledgeQueryResult,
  KnowledgeService,
} from "./interface";

/**
 * In-memory knowledge store with naive keyword-overlap scoring. This
 * stands in for a knowledge graph / embeddings-backed retrieval system;
 * the KnowledgeService interface is stable so a real backend can be
 * swapped in without changing consumers.
 */
export class LocalKnowledgeService implements KnowledgeService {
  private documents = new Map<string, KnowledgeDocument>();
  private counter = 0;
```

```
addDocument(input: {
  title: string;
  content: string;
  tags?: string[];
}): KnowledgeDocument {
  const doc: KnowledgeDocument = {
    id: `doc_${++this.counter}_${Date.now()}`,
    title: input.title,
    content: input.content,
    tags: input.tags ?? [],
    addedAt: new Date().toISOString(),
  };
  this.documents.set(doc.id, doc);
  return doc;
}

getDocument(id: string): KnowledgeDocument | undefined {
  return this.documents.get(id);
}

removeDocument(id: string): boolean {
  return this.documents.delete(id);
}

query(text: string, limit = 5): KnowledgeQueryResult[] {
  const queryWords = new Set(
    text.toLowerCase().split(/\W+/).filter(Boolean)
  );

  const scored: KnowledgeQueryResult[] = Array.from(
    this.documents.values()
  ).map((document) => {
    const docWords = `${document.title} ${document.content}`
      .toLowerCase()
      .split(/\W+/)
      .filter(Boolean);
    const overlap = docWords.filter((w) => queryWords.has(w)).length;
    const score = docWords.length ? overlap / docWords.length : 0;
    return { document, score };
  });

  return scored
    .filter((r) => r.score > 0)
    .sort((a, b) => b.score - a.score)
    .slice(0, limit);
}
```

```
  size(): number {
    return this.documents.size;
  }
}
```

src/lib/spark/intelligence/search/interface.ts

```
export interface SearchResult {
  id: string;
  title: string;
  snippet: string;
  score: number;
}

export interface SearchService {
  search(query: string, limit?: number): Promise<SearchResult[]>;
}
```

src/lib/spark/intelligence/search/local.ts

```
import type { SearchService, SearchResult } from "./interface";
import type { KnowledgeService } from "../../knowledge/interface";

/**
 * Search is a thin, focused layer over Knowledge retrieval. It depends
 * on the KnowledgeService abstraction rather than reaching into Memory
 * directly, keeping the dependency graph shallow: Search -> Knowledge.
 */
export class LocalSearchService implements SearchService {
  constructor(private readonly knowledge: KnowledgeService) {}

  async search(query: string, limit = 5): Promise<SearchResult[]> {
    const matches = this.knowledge.query(query, limit);
    return matches.map((m) => ({
      id: m.document.id,
      title: m.document.title,
      snippet: m.document.content.slice(0, 160),
      score: m.score,
    }));
  }
}
```

src/lib/spark/intelligence/recommendations/interface.ts

```
export interface Recommendation {
  itemId: string;
  reason: string;
  score: number;
}

export interface RecommendationsService {
  recommend(input: {
    userId?: string;
    sessionId?: string;
    candidateItems: Array<{ id: string; tags?: string[] }>;
    basedOnTags?: string[];
    limit?: number;
  }): Recommendation[];
}
```

src/lib/spark/intelligence/recommendations/local.ts

```
import type { Recommendation, RecommendationsService } from "./interface";

/**
 * Deterministic tag-overlap recommender. No model call required; this
 * can be replaced with a learned ranker later behind the same interface.
 */
export class LocalRecommendationsService implements RecommendationsService {
  recommend(input: {
    userId?: string;
    sessionId?: string;
    candidateItems: Array<{ id: string; tags?: string[] }>;
    basedOnTags?: string[];
    limit?: number;
  }): Recommendation[] {
    const wanted = new Set(input.basedOnTags ?? []);

    const scored: Recommendation[] = input.candidateItems.map((item) => {
      const tags = item.tags ?? [];
      const overlap = tags.filter((t) => wanted.has(t)).length;
      const score = wanted.size ? overlap / wanted.size : 0;
```

```
      return {
        itemId: item.id,
        reason: overlap
          ? `Matches ${overlap} of ${wanted.size} preferred tag(s).`
          : "No tag overlap; ranked by default order.",
        score,
      };
    });

    return scored
      .sort((a, b) => b.score - a.score)
      .slice(0, input.limit ?? 10);
  }
}
```

src/lib/spark/intelligence/organization/interface.ts

```
export interface OrganizableItem {
  id: string;
  label: string;
  tags?: string[];
  createdAt?: string;
}

export interface OrganizedGroup {
  key: string;
  items: OrganizableItem[];
}

export interface OrganizationService {
  groupByTag(items: OrganizableItem[]): OrganizedGroup[];
  sortByRecency(items: OrganizableItem[]): OrganizableItem[];
}
```

src/lib/spark/intelligence/organization/local.ts

```
import type {
  OrganizableItem,
  OrganizedGroup,
  OrganizationService,
} from "./interface";
```

```
/** Deterministic local organization logic — grouping and sorting only. */
export class LocalOrganizationService implements OrganizationService {
  groupByTag(items: OrganizableItem[]): OrganizedGroup[] {
    const groups = new Map<string, OrganizableItem[]>();
    for (const item of items) {
      const tags = item.tags?.length ? item.tags : ["untagged"];
      for (const tag of tags) {
        if (!groups.has(tag)) groups.set(tag, []);
        groups.get(tag)!.push(item);
      }
    }
    return Array.from(groups.entries()).map(([key, groupItems]) => ({
      key,
      items: groupItems,
    }));
  }

  sortByRecency(items: OrganizableItem[]): OrganizableItem[] {
    return [...items].sort((a, b) => {
      const aTime = a.createdAt ? new Date(a.createdAt).getTime() : 0;
      const bTime = b.createdAt ? new Date(b.createdAt).getTime() : 0;
      return bTime - aTime;
    });
  }
}
```

src/lib/spark/intelligence/personalization/interface.ts

```
export interface UserPreference {
  key: string;
  value: unknown;
  updatedAt: string;
}

export interface PersonalizationService {
  setPreference(userId: string, key: string, value: unknown): UserPreference;
  getPreference(userId: string, key: string): UserPreference | undefined;
  getAllPreferences(userId: string): UserPreference[];
}
```

src/lib/spark/intelligence/personalization/local.ts

```
import type { PersonalizationService, UserPreference } from "./interface";

/** In-memory per-user preference store. */
export class LocalPersonalizationService implements PersonalizationService {
  private preferences = new Map<string, Map<string, UserPreference>>();

  setPreference(userId: string, key: string, value: unknown): UserPreference {
    if (!this.preferences.has(userId)) {
      this.preferences.set(userId, new Map());
    }
    const pref: UserPreference = { key, value, updatedAt: new Date().toISOString(
    this.preferences.get(userId)!.set(key, pref);
    return pref;
  }

  getPreference(userId: string, key: string): UserPreference | undefined {
    return this.preferences.get(userId)?.get(key);
  }

  getAllPreferences(userId: string): UserPreference[] {
    return Array.from(this.preferences.get(userId)?.values() ?? []);
  }
}
```

src/lib/spark/intelligence/writing/interface.ts

```
export type WritingTone = "neutral" | "formal" | "casual" | "concise";

export interface WritingRequest {
  prompt: string;
  tone?: WritingTone;
  maxLength?: number;
}

export interface WritingResult {
  text: string;
  tone: WritingTone;
  provider: string;
}

export interface WritingService {
```

```
  draft(request: WritingRequest): Promise<WritingResult>;
}
```

src/lib/spark/intelligence/writing/local.ts

```
import type { WritingRequest, WritingResult, WritingService } from "./interface";
import type { ProviderRegistry } from "../../providers/registry";

/** Delegates drafting to the resolved AI provider (mock by default). */
export class LocalWritingService implements WritingService {
  constructor(private readonly providers: ProviderRegistry) {}

  async draft(request: WritingRequest): Promise<WritingResult> {
    const provider = this.providers.resolve();
    const tone = request.tone ?? "neutral";

    const completion = await provider.complete({
      messages: [
        {
          role: "system",
          content: `You are Spark's writing module. Tone: ${tone}.`,
        },
        { role: "user", content: request.prompt },
      ],
      maxTokens: request.maxLength,
    });

    return { text: completion.text, tone, provider: completion.provider };
  }
}
```

src/lib/spark/intelligence/translation/interface.ts

```
export interface TranslationRequest {
  text: string;
  targetLocale: string;
  sourceLocale?: string;
}

export interface TranslationResult {
  text: string;
```

```
  targetLocale: string;
  sourceLocale?: string;
  provider: string;
}

export interface TranslationService {
  translate(request: TranslationRequest): Promise<TranslationResult>;
}
```

src/lib/spark/intelligence/translation/local.ts

```
import type {
  TranslationRequest,
  TranslationResult,
  TranslationService,
} from "./interface";
import type { ProviderRegistry } from "../../providers/registry";

/** Delegates translation to the resolved AI provider (mock by default). */
export class LocalTranslationService implements TranslationService {
  constructor(private readonly providers: ProviderRegistry) {}

  async translate(request: TranslationRequest): Promise<TranslationResult> {
    const provider = this.providers.resolve();

    const completion = await provider.complete({
      messages: [
        {
          role: "system",
          content: `You are Spark's translation module. Translate into ${request.
        },
        { role: "user", content: request.text },
      ],
    });

    return {
      text: completion.text,
      targetLocale: request.targetLocale,
      sourceLocale: request.sourceLocale,
      provider: completion.provider,
    };
  }
}
```

src/lib/spark/intelligence/summaries/interface.ts

```
export interface SummaryRequest {
  text: string;
  maxSentences?: number;
}

export interface SummaryResult {
  summary: string;
  originalLength: number;
  summaryLength: number;
}

export interface SummariesService {
  summarize(request: SummaryRequest): SummaryResult;
}
```

src/lib/spark/intelligence/summaries/local.ts

```
import type { SummaryRequest, SummaryResult, SummariesService } from "./interface

/**
 * Deterministic extractive summarizer (first-N-sentences heuristic).
 * No AI provider required. Can be upgraded to abstractive summarization
 * via a provider later without changing the interface.
 */
export class LocalSummariesService implements SummariesService {
  summarize(request: SummaryRequest): SummaryResult {
    const sentences = request.text
      .split(/(?<=[.!?])\s+/)
      .map((s) => s.trim())
      .filter(Boolean);

    const maxSentences = request.maxSentences ?? 3;
    const summary = sentences.slice(0, maxSentences).join(" ");

    return {
      summary,
      originalLength: request.text.length,
      summaryLength: summary.length,
    };
  }
```

```
}
```

src/lib/spark/trust/privacy/interface.ts

```
export interface PiiFinding {
  type: "email" | "phone" | "ssn" | "credit_card";
  match: string;
  index: number;
}

export interface PrivacyService {
  scanForPii(text: string): PiiFinding[];
  redact(text: string): string;
}
```

src/lib/spark/trust/privacy/local.ts

```
import type { PiiFinding, PrivacyService } from "./interface";

const PATTERNS: Array<{ type: PiiFinding["type"]; regex: RegExp }> = [
  { type: "email", regex: /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g },
  { type: "phone", regex: /\+?\d[\d\s().-]{7,}\d/g },
  { type: "ssn", regex: /\b\d{3}-\d{2}-\d{4}\b/g },
  { type: "credit_card", regex: /\b(?:\d[ -]*?){13,16}\b/g },
];

/** Local, regex-based PII detection and redaction. No network calls. */
export class LocalPrivacyService implements PrivacyService {
  scanForPii(text: string): PiiFinding[] {
    const findings: PiiFinding[] = [];
    for (const { type, regex } of PATTERNS) {
      let match: RegExpExecArray | null;
      const re = new RegExp(regex);
      while ((match = re.exec(text)) !== null) {
        findings.push({ type, match: match[0], index: match.index });
        if (!re.global) break;
      }
    }
    return findings;
  }
```

```
  redact(text: string): string {
    let redacted = text;
    for (const { regex } of PATTERNS) {
      redacted = redacted.replace(new RegExp(regex), "[REDACTED]");
    }
    return redacted;
  }
}
```

src/lib/spark/trust/security/interface.ts

```
export interface SecurityCheckResult {
  safe: boolean;
  reasons: string[];
}

export interface SecurityService {
  /** Basic heuristic scan for common injection/scam patterns. */
  checkInput(text: string): SecurityCheckResult;
}
```

src/lib/spark/trust/security/local.ts

```
import type { SecurityCheckResult, SecurityService } from "./interface";

const SUSPICIOUS_PATTERNS: Array<{ label: string; regex: RegExp }> = [
  { label: "possible script injection", regex: /<script[\s>]/i },
  { label: "possible SQL injection", regex: /(\bunion\b.*\bselect\b|;\s*drop\s+ta
  { label: "urgent wire-transfer scam language", regex: /wire (transfer|money) im
  { label: "credential phishing language", regex: /verify your (password|account)
];

/** Local heuristic security scanner. No network calls, no ML model. */
export class LocalSecurityService implements SecurityService {
  checkInput(text: string): SecurityCheckResult {
    const reasons = SUSPICIOUS_PATTERNS.filter((p) => p.regex.test(text)).map(
      (p) => p.label
    );
    return { safe: reasons.length === 0, reasons };
  }
}
```

src/lib/spark/automation/interface.ts

```
export interface AutomationTask {
  id: string;
  name: string;
  status: "pending" | "running" | "completed" | "failed";
  result?: unknown;
  error?: string;
}

export type AutomationHandler = (input?: unknown) => Promise<unknown>;

export interface AutomationService {
  registerTask(name: string, handler: AutomationHandler): void;
  run(name: string, input?: unknown): Promise<AutomationTask>;
  history(limit?: number): AutomationTask[];
}
```

src/lib/spark/automation/local.ts

```
import type {
  AutomationHandler,
  AutomationService,
  AutomationTask,
} from "./interface";

/**
 * In-process workflow/task runner. Registers named handlers and
 * executes them on demand, tracking status/result/error for diagnostics.
 * No external orchestration system is required at this stage.
 */
export class LocalAutomationService implements AutomationService {
  private handlers = new Map<string, AutomationHandler>();
  private tasks: AutomationTask[] = [];
  private counter = 0;

  registerTask(name: string, handler: AutomationHandler): void {
    this.handlers.set(name, handler);
  }
```

```
  async run(name: string, input?: unknown): Promise<AutomationTask> {
    const handler = this.handlers.get(name);
    const task: AutomationTask = {
      id: `task_${++this.counter}_${Date.now()}`,
      name,
      status: "pending",
    };
    this.tasks.push(task);

    if (!handler) {
      task.status = "failed";
      task.error = `No automation task registered under name "${name}".`;
      return task;
    }

    task.status = "running";
    try {
      task.result = await handler(input);
      task.status = "completed";
    } catch (err) {
      task.status = "failed";
      task.error = err instanceof Error ? err.message : String(err);
    }
    return task;
  }

  history(limit = 50): AutomationTask[] {
    return this.tasks.slice(-limit);
  }
}
```

src/lib/spark/learning/interface.ts

```
export interface FeedbackEntry {
  id: string;
  subject: string;
  rating: "positive" | "negative" | "neutral";
  note?: string;
  createdAt: string;
}

export interface LearningService {
  recordFeedback(input: {
    subject: string;
```

```
    rating: "positive" | "negative" | "neutral";
    note?: string;
  }): FeedbackEntry;

  getFeedback(subject: string): FeedbackEntry[];

  /** Simple aggregate score in [-1, 1] based on recorded feedback. */
  score(subject: string): number;
}
```

src/lib/spark/learning/local.ts

```
import type { FeedbackEntry, LearningService } from "./interface";

const RATING_WEIGHT: Record<FeedbackEntry["rating"], number> = {
  positive: 1,
  neutral: 0,
  negative: -1,
};

/**
 * In-memory feedback store with a simple aggregate scoring function.
 * This is the seed of continuous improvement / adaptive personalization —
 * it does not train any model, it just tracks signal for later use.
 */
export class LocalLearningService implements LearningService {
  private feedback: FeedbackEntry[] = [];
  private counter = 0;

  recordFeedback(input: {
    subject: string;
    rating: FeedbackEntry["rating"];
    note?: string;
  }): FeedbackEntry {
    const entry: FeedbackEntry = {
      id: `fb_${++this.counter}_${Date.now()}`,
      subject: input.subject,
      rating: input.rating,
      note: input.note,
      createdAt: new Date().toISOString(),
    };
    this.feedback.push(entry);
    return entry;
  }
```

```
  getFeedback(subject: string): FeedbackEntry[] {
    return this.feedback.filter((f) => f.subject === subject);
  }

  score(subject: string): number {
    const entries = this.getFeedback(subject);
    if (!entries.length) return 0;
    const total = entries.reduce((sum, e) => sum + RATING_WEIGHT[e.rating], 0);
    return total / entries.length;
  }
}
```

src/lib/spark/providers/types.ts

```
/**
 * AI Provider abstraction.
 *
 * Spark services never call a provider SDK directly — they only ever
 * talk to this interface. Concrete providers (OpenAI, Anthropic, Gemini,
 * local models, etc.) are adapters that implement it and are registered
 * through the ProviderRegistry.
 */

export interface AIProviderMessage {
  role: "system" | "user" | "assistant";
  content: string;
}

export interface AIProviderCompletionRequest {
  messages: AIProviderMessage[];
  maxTokens?: number;
  temperature?: number;
}

export interface AIProviderCompletionResult {
  text: string;
  provider: string;
  model?: string;
  usage?: {
    inputTokens?: number;
    outputTokens?: number;
  };
}
```

```
export interface AIProviderEmbeddingResult {
  vector: number[];
  provider: string;
  model?: string;
}

export interface AIProvider {
  /** Unique identifier, e.g. "mock", "openai", "anthropic" */
  readonly name: string;

  /** Whether this provider is currently usable (e.g. has credentials). */
  isAvailable(): boolean;

  complete(
    request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult>;

  embed?(text: string): Promise<AIProviderEmbeddingResult>;
}
```

src/lib/spark/providers/registry.ts

```
import type { AIProvider } from "./types";
import { MockProvider } from "./mock";

/**
 * Holds all registered AI providers and resolves which one to use.
 * Services depend on this registry (or on a single resolved AIProvider),
 * never on a concrete vendor class.
 */
export class ProviderRegistry {
  private providers = new Map<string, AIProvider>();
  private defaultName: string;

  constructor() {
    const mock = new MockProvider();
    this.providers.set(mock.name, mock);
    this.defaultName = mock.name;
  }

  register(provider: AIProvider, makeDefault = false): void {
    this.providers.set(provider.name, provider);
    if (makeDefault) {
```

```
      this.defaultName = provider.name;
    }
  }

  setDefault(name: string): void {
    if (!this.providers.has(name)) {
      throw new Error(`Cannot set default: provider "${name}" is not registered.`
    }
    this.defaultName = name;
  }

  get(name?: string): AIProvider {
    const key = name ?? this.defaultName;
    const provider = this.providers.get(key);
    if (!provider) {
      throw new Error(`Provider "${key}" is not registered.`);
    }
    return provider;
  }

  /** Returns the default provider if available, else falls back to mock. */
  resolve(): AIProvider {
    const preferred = this.providers.get(this.defaultName);
    if (preferred && preferred.isAvailable()) {
      return preferred;
    }
    return this.providers.get("mock")!;
  }

  list(): Array<{ name: string; available: boolean; isDefault: boolean }> {
    return Array.from(this.providers.values()).map((p) => ({
      name: p.name,
      available: p.isAvailable(),
      isDefault: p.name === this.defaultName,
    }));
  }
}
```

src/lib/spark/providers/mock.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
```

```
  AIProviderEmbeddingResult,
} from "./types";

/**
 * Deterministic, offline provider. This is the default provider so that
 * Spark builds and runs with zero external dependencies and zero API keys.
 * It never makes a network call.
 */
export class MockProvider implements AIProvider {
  readonly name = "mock";

  isAvailable(): boolean {
    return true;
  }

  async complete(
    request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    const lastUser = [...request.messages]
      .reverse()
      .find((m) => m.role === "user");

    return {
      text: `[mock:${this.name}] acknowledged ${
        lastUser ? lastUser.content.length : 0
      } chars of input.`,
      provider: this.name,
      model: "mock-1",
      usage: { inputTokens: 0, outputTokens: 0 },
    };
  }

  async embed(text: string): Promise<AIProviderEmbeddingResult> {
    // Deterministic pseudo-embedding derived from character codes, so
    // the same input always yields the same vector without any model.
    const dims = 16;
    const vector = new Array(dims).fill(0);
    for (let i = 0; i < text.length; i++) {
      vector[i % dims] += text.charCodeAt(i);
    }
    const max = Math.max(1, ...vector.map(Math.abs));
    return {
      vector: vector.map((v) => v / max),
      provider: this.name,
      model: "mock-embed-1",
    };
  }
```

```
}
```

src/lib/spark/providers/openai.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
} from "./types";

/**
 * Placeholder adapter for OpenAI. Contains no SDK dependency and makes
 * no network calls. Swap in a real implementation later without
 * changing any Spark service code — they only depend on AIProvider.
 */
export class OpenAIProvider implements AIProvider {
  readonly name = "openai";

  constructor(private readonly apiKey?: string) {}

  isAvailable(): boolean {
    return Boolean(this.apiKey);
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "OpenAIProvider is a placeholder and has no API key configured."
      );
    }
    throw new Error("OpenAIProvider.complete() is not yet implemented.");
  }
}
```

src/lib/spark/providers/anthropic.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
```

```
  AIProviderCompletionResult,
} from "./types";

/**
 * Placeholder adapter for Anthropic. Contains no SDK dependency and
 * makes no network calls.
 */
export class AnthropicProvider implements AIProvider {
  readonly name = "anthropic";

  constructor(private readonly apiKey?: string) {}

  isAvailable(): boolean {
    return Boolean(this.apiKey);
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "AnthropicProvider is a placeholder and has no API key configured."
      );
    }
    throw new Error("AnthropicProvider.complete() is not yet implemented.");
  }
}
```

src/lib/spark/providers/gemini.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
} from "./types";

/** Placeholder adapter for Google Gemini. No SDK dependency, no network calls. *
export class GeminiProvider implements AIProvider {
  readonly name = "gemini";

  constructor(private readonly apiKey?: string) {}

  isAvailable(): boolean {
    return Boolean(this.apiKey);
```

```
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "GeminiProvider is a placeholder and has no API key configured."
      );
    }
    throw new Error("GeminiProvider.complete() is not yet implemented.");
  }
}
```

src/lib/spark/providers/local.ts

```
import type {
  AIProvider,
  AIProviderCompletionRequest,
  AIProviderCompletionResult,
} from "./types";

/**
 * Placeholder adapter for a local/self-hosted model (e.g. Ollama, a
 * local llama.cpp server). No network calls are made in this phase.
 */
export class LocalModelProvider implements AIProvider {
  readonly name = "local";

  constructor(private readonly endpoint?: string) {}

  isAvailable(): boolean {
    return Boolean(this.endpoint);
  }

  async complete(
    _request: AIProviderCompletionRequest
  ): Promise<AIProviderCompletionResult> {
    if (!this.isAvailable()) {
      throw new Error(
        "LocalModelProvider is a placeholder and has no endpoint configured."
      );
    }
    throw new Error("LocalModelProvider.complete() is not yet implemented.");
```

```
  }
}
```

Bud — Reusable Companion Layer

scripts/bud-verify.ts

```
import { createSpark } from "../src/lib/spark";
import { createBud } from "../src/lib/bud";

async function main() {
  const spark = createSpark();
  await spark.initialize();

  spark.knowledge.addDocument({
    title: "Refund Policy",
    content: "Refunds are issued within 5 business days of a return request.",
    tags: ["policy", "refunds"],
  });

  const bud = createBud({ spark, personality: { name: "Bud", tone: "concise" } })

  const session = { sessionId: "s1", userId: "u1", product: "unibud" };

  console.log("--- Turn 1: simple question ---");
  const r1 = await bud.respond("What is your refund policy?", session);
  console.log(r1);

  console.log("\n--- Turn 2: multi-step message (should trigger planning) ---");
  const r2 = await bud.respond(
    "Check my refund status then email me a confirmation",
    session
  );
  console.log(r2);
  if (r2.trace.plannedTaskCount < 2) {
    throw new Error("Expected planIfNeeded to detect a multi-step request.");
  }

  console.log("\n--- Turn 3: memory should now include prior turns ---");
  const r3 = await bud.respond("What did I just ask you?", session);
  console.log(r3);
  if (r3.trace.memoryHits === 0) {
    throw new Error("Expected memory recall to find prior turns.");
```

```
  }

  console.log("\n--- Transcript (read back purely from Spark memory) ---");
  const transcript = bud.transcript(session.sessionId);
  console.log(JSON.stringify(transcript, null, 2));
  if (transcript.length < 2) {
    throw new Error("Expected at least two stored conversation turns.");
  }

  console.log("\n--- Spark health after Bud usage ---");
  console.log(spark.health());

  await spark.shutdown();
  console.log("\nBud verification completed successfully.");
}

main().catch((err) => {
  console.error("Bud verification failed:", err);
  throw err;
});
```

src/lib/bud/README.md

```
# Bud — the reusable companion layer

Bud is a thin orchestration layer that sits between a user and Spark. It
holds a personality and a conversation flow — nothing else. All memory,
reasoning, planning, search, and text generation come from Spark.

```ts
import { createBud } from "@/lib/bud";

const bud = createBud();

const response = await bud.respond("What's my next task, then remind me to email
  sessionId: "session_1",
  userId: "user_1",
  product: "unibud",
});

console.log(response.message);
console.log(response.trace); // { memoryHits, knowledgeHits, reasoningConfidence,
```

Consumers should only ever `import { createBud } from "@/lib/bud"` **. Nothing under**
```

`bud/orchestrator.ts`, `bud/actions/*`, `bud/adapters/*`, etc. is a supported import path.

What Bud does NOT do

- Bud does not remember anything itself — `spark.memory` does.
- Bud does not search or retrieve anything itself — `spark.search` does.
- Bud does not reason about the message itself — `spark.reasoning` does.
- Bud does not plan multi-step goals itself — `spark.planning` does, once Bud has mechanically noticed the message looks like several steps.
- Bud does not call an AI provider, ever — `spark.writing` does, and only Spark decides which provider handles that call.

See `ARCHITECTURE.md` for the full request flow and the reasoning behind each of these boundaries.

Sharing a Spark instance

By default `createBud()` creates its own private Spark instance. If you want multiple Bud instances (or Bud plus another future assistant) to share memory/knowledge/identity, pass one in:

```
import { createSpark } from "@/lib/spark";
import { createBud } from "@/lib/bud";

const spark = createSpark();
await spark.initialize();

const bud = createBud({ spark });
```

Personality

Personality is plain configuration — a name, tone, and a few voice guidelines folded into the system prompt Bud sends to Spark's writing service. It contains no logic:

```
const bud = createBud({
  personality: {
    name: "Bud",
    tone: "concise",
    voice: ["Always suggest one concrete next step."],
```

```
  },
});
```

Reusability

Bud has no reference to UNIBUD, My Realm, Orbit, or any other product. The `product` field on `BudSession` is passed straight through to Spark's identity/context services — Bud doesn't interpret it. Any product can use the same Bud module by supplying its own sessions.

```
## `src/lib/bud/ARCHITECTURE.md`

```md
# Bud — Architecture

## Position in the ecosystem
```

User | ▼ Bud (personality + orchestration only) | ▼ Spark (all intelligence: memory, context, reasoning, planning, | knowledge, search, writing, translation, privacy, | security, automation, learning) ▼ AI Provider (Mock by default; OpenAI/Anthropic/Gemini/local when configured)

```
Bud never skips Spark to reach the AI provider layer directly, and Spark
never imports or knows about Bud.

## Directory layout
```

src/lib/bud/ ├── index.ts Public SDK — createBud(). The only supported import. ├── config.ts Static limits (recall/search result counts). ├── types.ts Session, message, and response shapes. ├── personality.ts Static tone/voice configuration. ├── orchestrator.ts The respond() flow, step by step. ├── conversation.ts Transcript formatting over Spark memory (no storage of its own). ├── prompts/ | ├── systemPrompt.ts Builds the system prompt from personality config. | └── userPrompt.ts Builds the user-turn prompt from Spark's outputs. ├── actions/ | ├── recallMemory.ts Calls spark.memory.recall | ├── searchKnowledge.ts Calls spark.search.search | ├── reason.ts Calls spark.reasoning.analyze | ├── planIfNeeded.ts Mechanical step detection -> spark.planning.createPlan | ├── generateResponse.ts Calls spark.writing.draft | └── storeInteraction.ts Calls
```

spark.memory.remember ├── context/ | └── buildContext.ts Ensures spark.identity has a context for the session └── adapters/ ├── sparkPort.ts The narrow interface Bud is allowed to depend on └── liveSparkAdapter.ts Wraps a real Spark instance to satisfy that interface

```
## The respond() flow

`bud.respond(message, session)` runs exactly these steps, in order,
every time:

1. **Build context** — `context/buildContext.ts` asks Spark's Identity
   service whether this session already has a context; creates one if not.
2. **Recall memory** — `actions/recallMemory.ts` asks Spark for prior
   episodic memory tied to this session.
3. **Search knowledge** — `actions/searchKnowledge.ts` asks Spark's
   Search service (which itself only talks to Knowledge) for anything
   relevant to the message.
4. **Reason** — `actions/reason.ts` hands the message plus everything
   recalled/found to Spark's Reasoning service and gets back an answer,
   a confidence score, and an explainable step trace.
5. **Plan (conditionally)** — `actions/planIfNeeded.ts` does a
   mechanical check (keyword/punctuation split, not semantic
   understanding) for whether the message looks like multiple
   sequenced asks. If so, it hands the extracted steps to Spark's
   Planning service. If not, no plan is created and no planning call
   is made.
6. **Generate response** — `actions/generateResponse.ts` builds a
   prompt (system prompt from personality + user prompt from Spark's
   reasoning/memory/knowledge outputs) and calls `spark.writing.draft()`.
   Spark resolves whichever `AIProvider` is registered — Bud never
   knows or cares which one.
7. **Store the turn** — `actions/storeInteraction.ts` writes both the
   user's message and Bud's reply back into Spark memory as episodic
   records, tagged `user`/`bud`.

Every one of those seven steps is a call through `BudSparkPort` — the
orchestrator (`orchestrator.ts`) contains no branching logic that
depends on the *content* of the message beyond step 5's mechanical
split. It doesn't summarize, doesn't judge relevance, doesn't decide
what's true — it delegates all of that.

## Why `adapters/sparkPort.ts` exists

`BudSparkPort` is a deliberately narrow re-typing of Spark's public
surface — only `identity`, `memory`, `search`, `reasoning`, `planning`,
```

```
and `writing`. Two consequences:

- **It's structurally impossible for Bud's action files to reach a
  capability that isn't listed** — there is no `providers` field on
  the port, so Bud cannot call an `AIProvider` even by mistake.
- **Bud is independently testable.** Anything satisfying
  `BudSparkPort`'s shape can stand in for Spark in a test, without
  spinning up a real Spark instance. `adapters/liveSparkAdapter.ts` is
  the only file that touches the concrete `Spark` class.

## Where conversation history actually lives

`conversation.ts` has no storage of its own. `getTranscript()` reads
Spark's episodic memory for a session and reshapes matched
user/bud record pairs into `ConversationTurn[]`. If Spark's memory for
that session is cleared, the transcript is gone — there is no shadow
copy inside Bud.

## Product independence

`BudSession.product` is passed straight through to Spark's identity and
context services without Bud inspecting or branching on its value.
There is no `if (product === "unibud")` anywhere in this module, and
there must never be — that would turn Bud from a reusable companion
into a product-specific one. New products reuse this exact module by
supplying their own `product` string.

## What's deliberately not here yet

- No persistence beyond whatever Spark itself persists (currently
  in-memory, see the Spark scaffold's README).
- No streaming responses — `writing.draft()` returns a complete string.
- No multi-turn planning follow-through (Bud creates a plan via step 5
  but does not yet check `spark.planning.nextTask()` on later turns to
  walk through it turn-by-turn).
- No integration into UNIBUD, My Realm, Orbit, or any application.
```

src/lib/bud/index.ts

```
/**
 * Public Bud SDK.
 *
 * Consumers should only ever import from here:
 *
```

```
 *   import { createBud } from "@/lib/bud";
 *
 * Never import from bud/orchestrator, bud/actions/*, bud/adapters/*,
 * etc. directly — those are internal implementation detail.
 */

import { createSpark, type Spark } from "../spark";
import { createLiveSparkAdapter } from "./adapters/liveSparkAdapter";
import type { BudSparkPort } from "./adapters/sparkPort";
import { createPersonality, type BudPersonality } from "./personality";
import { resolveLimits, type BudLimits } from "./config";
import { orchestrateRespond } from "./orchestrator";
import { getTranscript } from "./conversation";
import type { BudResponse, BudSession, ConversationTurn } from "./types";

export interface BudConfig {
  /** Provide an existing Spark instance to share it across products; otherwise B
  spark?: Spark;
  personality?: Partial<BudPersonality>;
  limits?: Partial<BudLimits>;
}

export interface Bud {
  respond(message: string, session: BudSession): Promise<BudResponse>;
  transcript(sessionId: string, limit?: number): ConversationTurn[];
  readonly personality: BudPersonality;
}

export function createBud(config: BudConfig = {}): Bud {
  const spark = config.spark ?? createSpark();
  const sparkPort: BudSparkPort = createLiveSparkAdapter(spark);
  const personality = createPersonality(config.personality);
  const limits = resolveLimits(config.limits);

  return {
    personality,

    async respond(message: string, session: BudSession): Promise<BudResponse> {
      return orchestrateRespond(sparkPort, personality, limits, message, session)
    },

    transcript(sessionId: string, limit?: number): ConversationTurn[] {
      return getTranscript(sparkPort, sessionId, limit);
    },
  };
}
```

```
export type { BudSession, BudResponse, BudMessage, ConversationTurn } from "./typ
export type { BudPersonality } from "./personality";
export type { BudLimits } from "./config";
```

src/lib/bud/types.ts

```
/**
 * Bud never defines its own memory, reasoning, or knowledge types —
 * those belong to Spark. This file only defines the shapes Bud needs
 * for orchestration: sessions, messages, and the final response.
 */

export interface BudSession {
  sessionId: string;
  userId: string;
  /** The product Bud is embedded in (e.g. "unibud", "campus"). Passed through to
  product: string;
  locale?: string;
  timezone?: string;
}

export interface BudMessage {
  role: "user" | "bud";
  content: string;
  timestamp: string;
}

export interface BudResponse {
  message: string;
  sessionId: string;
  /** Surfaced for debugging/observability — never re-derived by Bud itself. */
  trace: {
    memoryHits: number;
    knowledgeHits: number;
    reasoningConfidence: number;
    plannedTaskCount: number;
    provider: string;
  };
}

export interface ConversationTurn {
  user: BudMessage;
  bud: BudMessage;
}
```

src/lib/bud/config.ts

```
/**
 * Static configuration only. No behavior lives here — these are just
 * the knobs the orchestrator reads.
 */

export interface BudLimits {
  /** Max prior memory records recalled per turn. */
  memoryRecallLimit: number;
  /** Max knowledge/search results considered per turn. */
  knowledgeSearchLimit: number;
}

export const DEFAULT_BUD_LIMITS: BudLimits = {
  memoryRecallLimit: 10,
  knowledgeSearchLimit: 5,
};

export interface BudConfigOptions {
  name?: string;
  limits?: Partial<BudLimits>;
}

export function resolveLimits(overrides?: Partial<BudLimits>): BudLimits {
  return { ...DEFAULT_BUD_LIMITS, ...overrides };
}
```

src/lib/bud/personality.ts

```
/**
 * Personality is pure configuration: a name, a tone, and a short
 * description used to build prompts. It contains no logic and makes
 * no decisions — it's data that prompts/systemPrompt.ts reads.
 */

import type { WritingTone } from "../spark/intelligence/writing/interface";

export interface BudPersonality {
  name: string;
```

```
  description: string;
  tone: WritingTone;
  /** Short first-person guidance folded into the system prompt. */
  voice: string[];
}

export const DEFAULT_BUD_PERSONALITY: BudPersonality = {
  name: "Bud",
  description: "A supportive, plain-spoken companion.",
  tone: "casual",
  voice: [
    "Keep answers short and easy to act on.",
    "Be encouraging without being saccharine.",
    "Say plainly when you don't know something.",
  ],
};

export function createPersonality(
  overrides?: Partial<BudPersonality>
): BudPersonality {
  return { ...DEFAULT_BUD_PERSONALITY, ...overrides };
}
```

src/lib/bud/orchestrator.ts

```
import type { BudSparkPort } from "./adapters/sparkPort";
import type { BudPersonality } from "./personality";
import type { BudLimits } from "./config";
import type { BudResponse, BudSession } from "./types";
import { buildContext } from "./context/buildContext";
import { recallMemory } from "./actions/recallMemory";
import { searchKnowledge } from "./actions/searchKnowledge";
import { reason } from "./actions/reason";
import { planIfNeeded } from "./actions/planIfNeeded";
import { generateResponse } from "./actions/generateResponse";
import { storeInteraction } from "./actions/storeInteraction";

/**
 * The entire orchestration Bud performs, in order:
 *
 *   1. Build conversation context (ensure Spark identity exists)
 *   2. Recall memory from Spark
 *   3. Search Spark's knowledge
 *   4. Ask Spark to reason over message + recalled facts
```

```
 *   5. Mechanically detect multi-step requests and hand planning to Spark
 *   6. Ask Spark to draft the final response text
 *   7. Store the turn back through Spark memory
 *
 * Every step above calls into `spark` (a BudSparkPort) and nothing else.
 * There is no branch here that reasons about content, generates text
 * itself, or calls a provider — that's the point.
 */
export async function orchestrateRespond(
  spark: BudSparkPort,
  personality: BudPersonality,
  limits: BudLimits,
  message: string,
  session: BudSession
): Promise<BudResponse> {
  const context = buildContext(spark, session);

  const memoryRecords = recallMemory(
    spark,
    context.sessionId,
    limits.memoryRecallLimit
  );

  const knowledgeResults = await searchKnowledge(
    spark,
    message,
    limits.knowledgeSearchLimit
  );

  const reasoning = await reason(spark, message, memoryRecords, knowledgeResults)

  const plan = planIfNeeded(spark, message);

  const writing = await generateResponse(spark, personality, {
    message,
    memoryRecords,
    knowledgeResults,
    reasoning,
  });

  storeInteraction(spark, {
    sessionId: context.sessionId,
    userId: context.userId,
    userMessage: message,
    budMessage: writing.text,
  });
```

```
  return {
    message: writing.text,
    sessionId: context.sessionId,
    trace: {
      memoryHits: memoryRecords.length,
      knowledgeHits: knowledgeResults.length,
      reasoningConfidence: reasoning.confidence,
      plannedTaskCount: plan?.tasks.length ?? 0,
      provider: writing.provider,
    },
  };
}
```

src/lib/bud/conversation.ts

```
import type { BudSparkPort } from "./adapters/sparkPort";
import type { MemoryRecord } from "../spark/memory/interface";
import type { ConversationTurn, BudMessage } from "./types";

/**
 * Bud keeps no conversation store of its own. This module only reads
 * back what Spark already has in episodic memory and reshapes it into
 * a transcript. If Spark's memory is cleared, the transcript is gone —
 * there is no separate copy living in Bud.
 */
export function getTranscript(
  spark: BudSparkPort,
  sessionId: string,
  limit = 50
): ConversationTurn[] {
  const records = spark.memory
    .recall({ sessionId, kind: "episodic", limit: limit * 2 })
    .slice()
    .reverse(); // recall() returns newest-first; transcripts read oldest-first

  const turns: ConversationTurn[] = [];
  let pendingUser: BudMessage | null = null;

  for (const record of records) {
    const message = toMessage(record);
    if (message.role === "user") {
      pendingUser = message;
    } else if (pendingUser) {
      turns.push({ user: pendingUser, bud: message });
```

```
      pendingUser = null;
    }
  }

  return turns;
}

function toMessage(record: MemoryRecord): BudMessage {
  const isUser = record.tags?.includes("user");
  const content = record.content.replace(/^(User|Bud):\s*/, "");
  return {
    role: isUser ? "user" : "bud",
    content,
    timestamp: record.createdAt,
  };
}
```

src/lib/bud/context/buildContext.ts

```
import type { BudSparkPort } from "../adapters/sparkPort";
import type { BudSession } from "../types";

export interface ConversationContext {
  sessionId: string;
  userId: string;
  product: string;
}

/**
 * Ensures Spark has an identity context for this session (creating one
 * on first contact) and returns the minimal context the rest of the
 * orchestration needs. Bud does not track identity itself — Spark owns
 * that state.
 */
export function buildContext(
  spark: BudSparkPort,
  session: BudSession
): ConversationContext {
  const existing = spark.identity.getContext(session.sessionId);

  if (!existing) {
    spark.identity.createContext({
      userId: session.userId,
      sessionId: session.sessionId,
```

```
      product: session.product,
      locale: session.locale,
      timezone: session.timezone,
    });
  }

  return {
    sessionId: session.sessionId,
    userId: session.userId,
    product: session.product,
  };
}
```

src/lib/bud/adapters/sparkPort.ts

```
/**
 * BudSparkPort is the ONLY thing Bud's orchestration code depends on —
 * never the concrete Spark class directly. This keeps Bud honest about
 * "orchestration only": if a capability isn't listed here, Bud cannot
 * reach for it, including AI providers, which are deliberately absent
 * from this port.
 *
 * It is a subset of Spark's public API, re-typed narrowly so Bud's
 * intent (recall memory, search knowledge, reason, plan, draft text,
 * store memory) is visible at the type level.
 */

import type { IdentityContext } from "../../spark/core/identity/interface";
import type { ReasoningResult } from "../../spark/core/reasoning/interface";
import type { Plan } from "../../spark/core/planning/interface";
import type { MemoryKind, MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";
import type { WritingResult, WritingTone } from "../../spark/intelligence/writing

export interface BudSparkPort {
  identity: {
    createContext(input: {
      userId: string;
      sessionId: string;
      product: string;
      locale?: string;
      timezone?: string;
    }): IdentityContext;
    getContext(sessionId: string): IdentityContext | undefined;
```

```
  };

  memory: {
    remember(input: {
      kind: MemoryKind;
      content: string;
      sessionId?: string;
      userId?: string;
      tags?: string[];
    }): MemoryRecord;
    recall(query: {
      sessionId?: string;
      userId?: string;
      kind?: MemoryKind;
      limit?: number;
    }): MemoryRecord[];
  };

  search: {
    search(query: string, limit?: number): Promise<SearchResult[]>;
  };

  reasoning: {
    analyze(input: {
      question: string;
      facts?: string[];
    }): Promise<ReasoningResult>;
  };

  planning: {
    createPlan(goal: string, taskTitles: string[]): Plan;
  };

  writing: {
    draft(input: {
      prompt: string;
      tone?: WritingTone;
      maxLength?: number;
    }): Promise<WritingResult>;
  };
}
```

src/lib/bud/adapters/liveSparkAdapter.ts

```
import type { Spark } from "../../spark";
import type { BudSparkPort } from "./sparkPort";

/**
 * Wraps a concrete Spark instance so Bud's orchestrator only ever sees
 * the narrow BudSparkPort surface. Swapping this file for a test double
 * (e.g. an in-memory fake) is the only thing needed to unit-test Bud
 * without a real Spark instance.
 */
export function createLiveSparkAdapter(spark: Spark): BudSparkPort {
  return {
    identity: {
      createContext: (input) => spark.identity.createContext(input),
      getContext: (sessionId) => spark.identity.getContext(sessionId),
    },
    memory: {
      remember: (input) => spark.memory.remember(input),
      recall: (query) => spark.memory.recall(query),
    },
    search: {
      search: (query, limit) => spark.search.search(query, limit),
    },
    reasoning: {
      analyze: (input) => spark.reasoning.analyze(input),
    },
    planning: {
      createPlan: (goal, taskTitles) => spark.planning.createPlan(goal, taskTitle
    },
    writing: {
      draft: (input) => spark.writing.draft(input),
    },
  };
}
```

src/lib/bud/actions/recallMemory.ts

```
import type { BudSparkPort } from "../adapters/sparkPort";
import type { MemoryRecord } from "../../spark/memory/interface";

/**
 * Thin wrapper around Spark's memory recall. Bud does not implement
 * any memory logic itself — it only asks Spark for what it remembers.
 */
```

```
export function recallMemory(
  spark: BudSparkPort,
  sessionId: string,
  limit: number
): MemoryRecord[] {
  return spark.memory.recall({ sessionId, limit });
}
```

src/lib/bud/actions/searchKnowledge.ts

```
import type { BudSparkPort } from "../adapters/sparkPort";
import type { SearchResult } from "../../spark/intelligence/search/interface";

/**
 * Thin wrapper around Spark's search service. Bud never implements its
 * own search or retrieval — it only asks Spark.
 */
export async function searchKnowledge(
  spark: BudSparkPort,
  query: string,
  limit: number
): Promise<SearchResult[]> {
  return spark.search.search(query, limit);
}
```

src/lib/bud/actions/reason.ts

```
import type { BudSparkPort } from "../adapters/sparkPort";
import type { ReasoningResult } from "../../spark/core/reasoning/interface";
import type { MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";

/**
 * Assembles the facts Spark's reasoning service should consider — from
 * recalled memory and search results — and delegates the actual
 * analysis to Spark. Bud does not reason on its own.
 */
export async function reason(
  spark: BudSparkPort,
  message: string,
  memoryRecords: MemoryRecord[],
```

```
  knowledgeResults: SearchResult[]
): Promise<ReasoningResult> {
  const facts = [
    ...memoryRecords.map((m) => m.content),
    ...knowledgeResults.map((k) => `${k.title}: ${k.snippet}`),
  ];

  return spark.reasoning.analyze({ question: message, facts });
}
```

src/lib/bud/actions/planIfNeeded.ts

```
import type { BudSparkPort } from "../adapters/sparkPort";
import type { Plan } from "../../spark/core/planning/interface";

const STEP_SEPARATORS = /\bthen\b|;|\n-|\n\d+\./i;

/**
 * This is mechanical text splitting, not reasoning about intent — Bud
 * detects a shape (multiple steps separated by "then"/semicolons/list
 * markers) and hands the actual planning off to Spark. If the message
 * doesn't look like a multi-step request, no plan is created.
 */
export function planIfNeeded(spark: BudSparkPort, message: string): Plan | null {
  if (!STEP_SEPARATORS.test(message)) {
    return null;
  }

  const steps = message
    .split(STEP_SEPARATORS)
    .map((s) => s.replace(/^[\s\-.\d]+/, "").trim())
    .filter((s) => s.length > 0);

  if (steps.length < 2) {
    return null;
  }

  return spark.planning.createPlan(message, steps);
}
```

src/lib/bud/actions/generateResponse.ts

```
import type { BudSparkPort } from "../adapters/sparkPort";
import type { BudPersonality } from "../personality";
import type { ReasoningResult } from "../../spark/core/reasoning/interface";
import type { MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";
import { buildSystemPrompt } from "../prompts/systemPrompt";
import { buildUserPrompt } from "../prompts/userPrompt";
import type { WritingResult } from "../../spark/intelligence/writing/interface";

/**
 * Delegates text generation to Spark.writing (which in turn delegates
 * to whatever AIProvider is configured). Bud never calls a provider
 * directly — this is the only path to generated text.
 */
export async function generateResponse(
  spark: BudSparkPort,
  personality: BudPersonality,
  input: {
    message: string;
    memoryRecords: MemoryRecord[];
    knowledgeResults: SearchResult[];
    reasoning: ReasoningResult;
  }
): Promise<WritingResult> {
  const system = buildSystemPrompt(personality);
  const user = buildUserPrompt(input);

  return spark.writing.draft({
    prompt: `${system}\n\n${user}`,
    tone: personality.tone,
  });
}
```

src/lib/bud/actions/storeInteraction.ts

```
import type { BudSparkPort } from "../adapters/sparkPort";

/**
 * Stores both sides of the exchange as episodic memory through Spark.
 * Bud holds no memory of its own — this is the only place a turn is
 * persisted, and it goes straight into Spark.
 */
export function storeInteraction(
  spark: BudSparkPort,
```

```
  input: {
    sessionId: string;
    userId: string;
    userMessage: string;
    budMessage: string;
  }
): void {
  spark.memory.remember({
    kind: "episodic",
    content: `User: ${input.userMessage}`,
    sessionId: input.sessionId,
    userId: input.userId,
    tags: ["conversation", "user"],
  });

  spark.memory.remember({
    kind: "episodic",
    content: `Bud: ${input.budMessage}`,
    sessionId: input.sessionId,
    userId: input.userId,
    tags: ["conversation", "bud"],
  });
}
```

src/lib/bud/prompts/systemPrompt.ts

```
import type { BudPersonality } from "../personality";

/**
 * Pure string assembly from static personality configuration. No
 * decision-making happens here — this is a template, not a model.
 */
export function buildSystemPrompt(personality: BudPersonality): string {
  return [
    `You are ${personality.name}. ${personality.description}`,
    ...personality.voice.map((line) => `- ${line}`),
  ].join("\n");
}
```

src/lib/bud/prompts/userPrompt.ts

```
import type { ReasoningResult } from "../../spark/core/reasoning/interface";
import type { MemoryRecord } from "../../spark/memory/interface";
import type { SearchResult } from "../../spark/intelligence/search/interface";

/**
 * Assembles the prompt handed to Spark.writing.draft(). All of the
 * "thinking" was already done by Spark (reasoning, search, memory) —
 * this just formats it into text for the writing step.
 */
export function buildUserPrompt(input: {
  message: string;
  memoryRecords: MemoryRecord[];
  knowledgeResults: SearchResult[];
  reasoning: ReasoningResult;
}): string {
  const { message, memoryRecords, knowledgeResults, reasoning } = input;

  const sections = [`User said: "${message}"`, `Spark's analysis: ${reasoning.ans

  if (memoryRecords.length) {
    sections.push(
      `Relevant memory:\n${memoryRecords.map((m) => `- ${m.content}`).join("\n")}
    );
  }

  if (knowledgeResults.length) {
    sections.push(
      `Relevant knowledge:\n${knowledgeResults
        .map((k) => `- ${k.title}: ${k.snippet}`)
        .join("\n")}`
    );
  }

  sections.push("Reply to the user directly, in one short response.");

  return sections.join("\n\n");
}
```

UNIBUD — Student Dashboard Product

scripts/unibud-verify.ts

```
import { createSpark } from "../src/lib/spark";
```

```
import { createBud } from "../src/lib/bud";
import { assembleDashboard, setGoals } from "../src/lib/unibud";

async function main() {
  const spark = createSpark();
  await spark.initialize();
  const bud = createBud({ spark, personality: { name: "Bud", tone: "casual" } });

  const session = { sessionId: "unibud_s1", userId: "student_1", product: "unibud

  console.log("--- Dashboard with no stated goals yet ---");
  const first = await assembleDashboard(spark, bud, session);
  console.log(JSON.stringify(first, null, 2));
  if (first.suggestedToolId !== null) {
    throw new Error("Expected no suggestion before any goals are set.");
  }

  console.log("\n--- Student sets a goal, asks a real question ---");
  setGoals(spark, session.userId, ["grades", "review"]);
  const second = await assembleDashboard(
    spark,
    bud,
    session,
    "I have a chemistry exam next week, what should I focus on?"
  );
  console.log(JSON.stringify(second, null, 2));

  if (!second.suggestedToolId) {
    throw new Error("Expected a suggested tool once goal tags are set.");
  }
  if (!["study_plans", "assignments"].includes(second.suggestedToolId)) {
    throw new Error(
      `Expected study_plans or assignments to rank highest for ["grades","review"
    );
  }

  console.log("\n--- Transcript is readable straight from Bud/Spark, UNIBUD store
  console.log(bud.transcript(session.sessionId));

  await spark.shutdown();
  console.log("\nUNIBUD dashboard verification completed successfully.");
}

main().catch((err) => {
  console.error("UNIBUD verification failed:", err);
  throw err;
});
```

src/lib/unibud/index.ts

```
export { assembleDashboard } from "./dashboard";
export { setGoals, getGoals } from "./goals";
export { TOOL_DEFINITIONS } from "./tools";
export type { ToolId, ToolDefinition, DashboardViewModel } from "./types";
```

src/lib/unibud/types.ts

```
/**
 * These types belong to UNIBUD only. Spark and Bud know nothing about
 * "tools," "goals," or a "dashboard" — those are UNIBUD's own concepts,
 * built on top of the frozen Spark/Bud SDKs.
 */

export type ToolId =
  | "assignments"
  | "projects"
  | "smart_notes"
  | "study_plans"
  | "campus"
  | "connect";

export interface ToolDefinition {
  id: ToolId;
  name: string;
  description: string;
  tags: string[];
}

export interface DashboardViewModel {
  greeting: string;
  tools: ToolDefinition[];
  suggestedToolId: ToolId | null;
  suggestedReason: string;
  trace: {
    memoryHits: number;
    knowledgeHits: number;
    reasoningConfidence: number;
    provider: string;
```

```
  };
}
```

src/lib/unibud/tools.ts

```
import type { ToolDefinition } from "./types";

/**
 * Static configuration, not intelligence. Tags here are matched against
 * a student's stored goal tags (via Spark's Personalization service) to
 * drive the "suggested next" pick on the dashboard.
 */
export const TOOL_DEFINITIONS: ToolDefinition[] = [
  {
    id: "assignments",
    name: "Assignments",
    description: "Track what's due and what's done.",
    tags: ["deadlines", "coursework", "grades"],
  },
  {
    id: "projects",
    name: "Projects",
    description: "Longer-running work with milestones.",
    tags: ["coursework", "collaboration", "deadlines"],
  },
  {
    id: "smart_notes",
    name: "Smart Notes",
    description: "Notes that connect back to your courses.",
    tags: ["coursework", "study", "review"],
  },
  {
    id: "study_plans",
    name: "Study Plans",
    description: "A sequenced plan for an exam or a subject.",
    tags: ["study", "review", "grades"],
  },
  {
    id: "campus",
    name: "Campus",
    description: "Events, deadlines, and services at your school.",
    tags: ["campus_life", "community"],
  },
  {
```

```
    id: "connect",
    name: "Connect",
    description: "Study groups, mentors, and classmates.",
    tags: ["community", "collaboration"],
  },
];
```

src/lib/unibud/goals.ts

```
import type { Spark } from "../spark";

const GOALS_KEY = "academic_goal_tags";

/**
 * UNIBUD-specific convenience over Spark.personalization. Spark itself
 * has no concept of "academic goals" — it just stores arbitrary tagged
 * preferences per user. This is where UNIBUD gives that generic
 * mechanism product meaning.
 */
export function setGoals(spark: Spark, userId: string, tags: string[]): void {
  spark.personalization.setPreference(userId, GOALS_KEY, tags);
}

export function getGoals(spark: Spark, userId: string): string[] {
  const pref = spark.personalization.getPreference(userId, GOALS_KEY);
  return Array.isArray(pref?.value) ? (pref!.value as string[]) : [];
}
```

src/lib/unibud/dashboard.ts

```
import type { Spark } from "../spark";
import type { Bud } from "../bud";
import type { BudSession } from "../bud";
import type { DashboardViewModel } from "./types";
import { TOOL_DEFINITIONS } from "./tools";
import { getGoals } from "./goals";

/**
 * The landing-page flow described in the product brief:
 *
 *   1. Bud greets the student and reasons over their message (via Bud's
```

```
 *      own respond() — UNIBUD does not reimplement conversation).
 *   2. Spark.recommendations ranks UNIBUD's own tool catalog against
 *      the student's stored goal tags — a dashboard-level concern Bud
 *      doesn't expose, so UNIBUD calls Spark directly for it.
 *
 * Bud and Spark's public APIs are used exactly as frozen; nothing here
 * reaches into their internals.
 */
export async function assembleDashboard(
  spark: Spark,
  bud: Bud,
  session: BudSession,
  message?: string
): Promise<DashboardViewModel> {
  const response = await bud.respond(
    message ?? "Greet me and help me figure out what to work on.",
    session
  );

  const goalTags = getGoals(spark, session.userId);

  const ranked = spark.recommendations.recommend({
    userId: session.userId,
    sessionId: session.sessionId,
    candidateItems: TOOL_DEFINITIONS.map((t) => ({ id: t.id, tags: t.tags })),
    basedOnTags: goalTags,
    limit: 1,
  });

  const top = ranked[0];
  const suggestedToolId = top && top.score > 0 ? (top.itemId as DashboardViewMode

  return {
    greeting: response.message,
    tools: TOOL_DEFINITIONS,
    suggestedToolId,
    suggestedReason: top?.reason ?? "Set an academic goal to get a personalized p
    trace: {
      memoryHits: response.trace.memoryHits,
      knowledgeHits: response.trace.knowledgeHits,
      reasoningConfidence: response.trace.reasoningConfidence,
      provider: response.trace.provider,
    },
  };
}
```

`unibud-dashboard.jsx` (interactive preview component)

```jsx
import React, { useMemo, useState } from "react";
import {
  BookOpen,
  FolderKanban,
  NotebookPen,
  CalendarClock,
  Building2,
  Users,
  Pin,
  Sparkles,
} from "lucide-react";

/**
 * UNIBUD — student dashboard (visual preview)
 * ---------------------------------------------------------------
 * This is a self-contained UI preview for the chat surface. The real
 * data behind it — Bud's greeting, the ranked "focus pick" — is
 * already implemented and verified against the actual Spark/Bud SDKs
 * in src/lib/unibud/dashboard.ts (assembleDashboard()). In production
 * this component would call that function through an API route
 * instead of the local mock logic below, which mirrors the same
 * tag-overlap scoring Spark.recommendations performs so the behavior
 * you see here matches the verified backend.
 */

const PALETTE = {
  ink: "#1B1F3B",
  paper: "#FBF8F2",
  blue: "#4C6FFF",
  coral: "#FF6B5E",
  sage: "#7C9473",
  graphite: "#2B2B2B",
  hair: "#E4DFD3",
};

const TOOLS = [
  {
    id: "assignments",
    name: "Assignments",
    description: "Track what's due and what's done.",
    tags: ["deadlines", "coursework", "grades"],
    icon: BookOpen,
    stat: "3 due this week",
  },
```

```
  {
    id: "projects",
    name: "Projects",
    description: "Longer-running work with milestones.",
    tags: ["coursework", "collaboration", "deadlines"],
    icon: FolderKanban,
    stat: "1 milestone Friday",
  },
  {
    id: "smart_notes",
    name: "Smart Notes",
    description: "Notes that connect back to your courses.",
    tags: ["coursework", "study", "review"],
    icon: NotebookPen,
    stat: "12 notes this term",
  },
  {
    id: "study_plans",
    name: "Study Plans",
    description: "A sequenced plan for an exam or a subject.",
    tags: ["study", "review", "grades"],
    icon: CalendarClock,
    stat: "Chem exam in 6 days",
  },
  {
    id: "campus",
    name: "Campus",
    description: "Events, deadlines, and services at your school.",
    tags: ["campus_life", "community"],
    icon: Building2,
    stat: "2 events this week",
  },
  {
    id: "connect",
    name: "Connect",
    description: "Study groups, mentors, and classmates.",
    tags: ["community", "collaboration"],
    icon: Users,
    stat: "1 group invite",
  },
];

const GOAL_OPTIONS = [
  { key: "grades", label: "Raise my grades" },
  { key: "review", label: "Get ready for exams" },
  { key: "deadlines", label: "Stop missing deadlines" },
  { key: "community", label: "Find people to work with" },
```

```
];

function rankTools(goalTags) {
  if (goalTags.length === 0) return null;
  const scored = TOOLS.map((tool) => {
    const overlap = tool.tags.filter((t) => goalTags.includes(t)).length;
    return { tool, score: overlap / goalTags.length };
  });
  scored.sort((a, b) => b.score - a.score);
  return scored[0].score > 0 ? scored[0] : null;
}

function greetingFor(name, goalTags) {
  const hour = new Date().getHours();
  const timeGreeting = hour < 12 ? "Morning" : hour < 18 ? "Afternoon" : "Evening
  if (goalTags.length === 0) {
    return `${timeGreeting}, ${name}. Tell me what you're working toward and I'll
  }
  return `${timeGreeting}, ${name}. Based on what you told me, here's where I'd s
}

export default function UnibudDashboard() {
  const [goalTags, setGoalTags] = useState([]);
  const [activeToolId, setActiveToolId] = useState(null);

  const toggleGoal = (key) => {
    setGoalTags((prev) =>
      prev.includes(key) ? prev.filter((k) => k !== key) : [...prev, key]
    );
  };

  const pick = useMemo(() => rankTools(goalTags), [goalTags]);
  const greeting = useMemo(() => greetingFor("Andrew", goalTags), [goalTags]);
  const activeTool = TOOLS.find((t) => t.id === activeToolId) || null;

  return (
    <div
      style={{
        minHeight: "100vh",
        background: PALETTE.paper,
        color: PALETTE.graphite,
        fontFamily: "'IBM Plex Sans', system-ui, sans-serif",
      }}
    >
      <style>{`
        @import url('https://fonts.googleapis.com/css2?family=Fraunces:wght@400;5
        .unibud-card { transition: transform 160ms ease, box-shadow 160ms ease; }
```

```
  .unibud-card:hover { transform: translateY(-3px); box-shadow: 0 10px 24px
  .unibud-pin { animation: unibud-pin-in 420ms cubic-bezier(.2,.9,.3,1.3);
  @keyframes unibud-pin-in {
    from { opacity: 0; transform: rotate(-6deg) translateY(-10px) scale(0.9
    to   { opacity: 1; transform: rotate(-2deg) translateY(0) scale(1); }
  }
  .unibud-chip { transition: background 140ms ease, color 140ms ease, borde
  @media (prefers-reduced-motion: reduce) {
    .unibud-card, .unibud-pin, .unibud-chip { transition: none; animation:
  }
`}</style>

{/* Header / Bud greeting */}
<header style={{ background: PALETTE.ink, padding: "28px 20px 40px" }}>
  <div style={{ maxWidth: 720, margin: "0 auto" }}>
    <div
      style={{
        display: "flex",
        alignItems: "center",
        gap: 8,
        marginBottom: 14,
      }}
    >
      <div
        style={{
          width: 30,
          height: 30,
          borderRadius: 999,
          background: PALETTE.coral,
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          flexShrink: 0,
        }}
      >
        <Sparkles size={16} color={PALETTE.ink} />
      </div>
      <span
        style={{
          fontFamily: "'IBM Plex Mono', monospace",
          fontSize: 12,
          letterSpacing: "0.08em",
          color: "#9AA3D1",
          textTransform: "uppercase",
        }}
      >
        Bud · UNIBUD
```

```
      </span>
    </div>
    <h1
      style={{
        fontFamily: "'Fraunces', serif",
        fontWeight: 500,
        fontSize: "clamp(22px, 5vw, 30px)",
        lineHeight: 1.3,
        color: PALETTE.paper,
        margin: 0,
      }}
    >
      {greeting}
    </h1>
  </div>
</header>

{/* Goal chips */}
<div style={{ maxWidth: 720, margin: "0 auto", padding: "20px 20px 0" }}>
  <p
    style={{
      fontFamily: "'IBM Plex Mono', monospace",
      fontSize: 11,
      letterSpacing: "0.06em",
      textTransform: "uppercase",
      color: "#8A8578",
      margin: "0 0 10px",
    }}
  >
    What are you working toward?
  </p>
  <div style={{ display: "flex", flexWrap: "wrap", gap: 8 }}>
    {GOAL_OPTIONS.map((g) => {
      const active = goalTags.includes(g.key);
      return (
        <button
          key={g.key}
          onClick={() => toggleGoal(g.key)}
          className="unibud-chip"
          style={{
            padding: "7px 14px",
            borderRadius: 999,
            fontSize: 13.5,
            fontFamily: "'IBM Plex Sans', sans-serif",
            fontWeight: 500,
            border: `1px solid ${active ? PALETTE.blue : PALETTE.hair}`,
            background: active ? PALETTE.blue : "transparent",
```

```
            color: active ? PALETTE.paper : PALETTE.graphite,
            cursor: "pointer",
          }}
        >
          {g.label}
        </button>
      );
    })}
  </div>
</div>

{/* Focus pick — the signature element */}
<div style={{ maxWidth: 720, margin: "0 auto", padding: "26px 20px 0" }}>
  {pick ? (
    <div
      key={pick.tool.id}
      className="unibud-pin"
      style={{
        transform: "rotate(-2deg)",
        background: "#FFF7DC",
        border: `1px solid ${PALETTE.hair}`,
        borderRadius: 4,
        padding: "16px 18px",
        boxShadow: "0 8px 20px rgba(27,31,59,0.10)",
        position: "relative",
        maxWidth: 420,
      }}
    >
      <Pin
        size={16}
        color={PALETTE.coral}
        style={{ position: "absolute", top: -8, left: 18, transform: "rotat
        fill={PALETTE.coral}
      />
      <p
        style={{
          fontFamily: "'IBM Plex Mono', monospace",
          fontSize: 11,
          letterSpacing: "0.06em",
          textTransform: "uppercase",
          color: PALETTE.coral,
          margin: "0 0 6px",
        }}
      >
        Bud's pick
      </p>
      <p style={{ margin: "0 0 4px", fontFamily: "'Fraunces', serif", fontS
```

```
        Start with {pick.tool.name}
      </p>
      <p style={{ margin: 0, fontSize: 13.5, color: "#5B5748" }}>
        {pick.tool.stat} — matches what you told me you're working on.
      </p>
    </div>
  ) : (
    <p style={{ fontSize: 13.5, color: "#8A8578", fontStyle: "italic" }}>
      Pick a goal above and I'll point you at one tool to start with.
    </p>
  )}
</div>

{/* Tool grid */}
<main style={{ maxWidth: 720, margin: "0 auto", padding: "26px 20px 60px" }
  <div
    style={{
      display: "grid",
      gridTemplateColumns: "repeat(auto-fit, minmax(150px, 1fr))",
      gap: 14,
    }}
  >
    {TOOLS.map((tool) => {
      const Icon = tool.icon;
      const isPick = pick?.tool.id === tool.id;
      const isActive = activeToolId === tool.id;
      return (
        <button
          key={tool.id}
          onClick={() => setActiveToolId(isActive ? null : tool.id)}
          className="unibud-card"
          style={{
            textAlign: "left",
            background: "#fff",
            border: `1px solid ${isPick ? PALETTE.blue : PALETTE.hair}`,
            borderRadius: 10,
            padding: "16px 16px 14px",
            cursor: "pointer",
            fontFamily: "inherit",
          }}
        >
          <Icon size={20} color={isPick ? PALETTE.blue : PALETTE.graphite}
          <p
            style={{
              margin: "10px 0 4px",
              fontFamily: "'Fraunces', serif",
              fontSize: 16.5,
```

```
              color: PALETTE.graphite,
            }}
          >
            {tool.name}
          </p>
          <p style={{ margin: "0 0 10px", fontSize: 12.5, color: "#7A7568",
            {tool.description}
          </p>
          <p
            style={{
              margin: 0,
              fontFamily: "'IBM Plex Mono', monospace",
              fontSize: 11,
              color: PALETTE.sage,
            }}
          >
            {tool.stat}
          </p>
        </button>
      );
    })}
  </div>

  {activeTool && (
    <div
      style={{
        marginTop: 18,
        borderTop: `1px solid ${PALETTE.hair}`,
        paddingTop: 16,
      }}
    >
      <p
        style={{
          fontFamily: "'IBM Plex Mono', monospace",
          fontSize: 11,
          letterSpacing: "0.06em",
          textTransform: "uppercase",
          color: "#8A8578",
          margin: "0 0 6px",
        }}
      >
        Opening
      </p>
      <p style={{ margin: 0, fontFamily: "'Fraunces', serif", fontSize: 18
        {activeTool.name} — not built yet in this preview.
      </p>
      <p style={{ margin: "6px 0 0", fontSize: 13, color: "#7A7568" }}>
```

```
              This is where the {activeTool.name.toLowerCase()} workspace opens n
            </p>
          </div>
        )}
      </main>
    </div>
  );
}
```